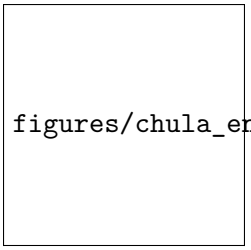
Capstone Design Project I



A Modular Self-Reconfigurable Robot (MSRR) for Search and Space Exploration

Submitted to the
Project Committee appointed by the
International School of Engineering (ISE)
Faculty of Engineering, Chulalongkorn University

Project Advisor

Asst. Prof. Paulo Fernando Rocha Garcia, Ph.D.

Submitted By

Kittivut Pipattanachaiyapong	6638019121
Krin Kaweewongsonthorn	6638027121
Pakapol Kijsupanont	6638172421
Phattharamon Sukvivatn	6638178221

1/2026: 2147498 Capstone Design Project I
Robotics and Artificial Intelligence Engineering (International Program)
International School of Engineering (ISE), Faculty of Engineering,
Chulalongkorn University

Abstract

Modular self-reconfigurable robots (MSRRs) offer a single set of hardware that changes its own body plan to suit whatever terrain it meets. That property is what makes them attractive for planetary surface exploration and for search and rescue: launch mass and volume are constrained, the environment cannot be mapped in advance, and a robot stopped by a crevasse or a crater-rim ledge is a robot that has ended its mission. Two decades of research have largely settled the *shape-oriented* problem — given a target configuration, compute and execute a reconfiguration sequence. Deployment requires the inverse, *task-oriented* framing: perceive the environment, infer the capability the obstacle demands, and derive the configuration. The closest existing demonstrations of that loop still depend on external motion capture, off-board computation, or a single dedicated sensor module, and learning-based reconfiguration remains almost entirely confined to simulation — precisely where contact dynamics and magnetic docking fail to transfer.

This project designs, builds and evaluates a number of mechanically homogeneous MSRR modules together with a physics-validated digital twin, and closes the perceive → select → reconfigure → traverse loop using onboard sensing and computation only. The module reverse-engineers the SMORES architecture: a four-degree-of-freedom hybrid chain/lattice/mobile body whose two side rotations serve interchangeably as differential-drive wheels and as articulated joints, fitted with genderless electropermanent (EP-Face) connectors that hold without standby power. Positioning is cascaded — ultra-wideband ranging fused with inertial measurement for macro-scale relative localisation, infrared sensing and IMU feedback for short-range alignment — and perception is deliberately low-SWaP, pairing a monocular camera with an 8×8 multi-zone time-of-flight sensor rather than an RGB-D camera. Reconfiguration planning inherits the SMORES-EP Kuhn–Munkres task-assignment and parallel self-assembly pipeline, extended with a prioritised multi-robot route planner and, above it, the configuration-selection layer that makes the system task-oriented rather than shape-oriented. Development is staged across four phases, from external perception and computation with contact-free locomotion, through docked reconfiguration, to fully onboard perception and computation. Success is measured against three obstacle classes — gap, step and narrow passage — with targets of at least 80 % configuration-selection success in simulation including held-out obstacle parameters, at least 90 % autonomous dock/undock over twenty hardware trials, and closure of the full autonomous loop on one obstacle class in hardware.

Keywords. self-assembling robot; modular robot; hierarchical control; onboard perception; SMORES-EP; autonomous docking; search and space exploration.

Contents

1 Introduction

A planetary rover is built once, launched once, and then meets terrain nobody has surveyed at the scale that matters to it. Orbital imagery resolves features of tens of metres; the things that end missions are smaller than that. A crevasse a little wider than the wheelbase, a crater-rim ledge a little taller than the wheel radius, the mouth of a lava tube narrower than the chassis — each is invisible from orbit and impassable on arrival. The conventional answer is to drive around, and where driving around is impossible the mission stops there. *Spirit* ended its operational life in soft regolith it could not reverse out of; the same class of problem, at the same scale, recurs wherever a fixed-shape vehicle meets unsurveyed ground.

The constraint behind this is not intelligence but geometry. A wheeled rover has one body plan, chosen years before launch against terrain models that are necessarily coarse. Whatever that plan cannot cross, it cannot cross. Sending several specialised vehicles instead multiplies launch mass and volume, the two resources a planetary mission has least of, and each vehicle remains a single point of failure for its own speciality.

Modular self-reconfigurable robots (MSRRs) attack the constraint directly. An MSRR is a set of identical modules, each able to drive on its own and to dock rigidly to its neighbours, so that the collective can change its own body plan in the field. Four properties make the architecture a good fit for space and for search and rescue specifically, rather than merely an interesting machine:

- **Shape change substitutes for route planning.** The same modules drive as independent wheeled units on open ground, chain into a bridge to span a gap, or form a cantilever arch to climb a ledge. Terrain that defeats one morphology is crossed by another.
- **Launch mass and volume are amortised.** One set of homogeneous modules covers the capabilities that would otherwise require several purpose-built vehicles, and identical modules pack more densely than articulated ones.
- **Redundancy is intrinsic.** Modules are interchangeable, so a failed unit is replaced from the remaining pool rather than ending the mission — the property demonstrated in self-repair after catastrophic damage [**?**].
- **Access scales below the vehicle envelope.** A single module can enter an opening the assembled robot cannot, then reassemble on the far side. Confined voids — lava tubes, collapsed structures, ducts — are reachable in a way no fixed chassis reaches them.

Two decades of research have made the mechanism work. Connectors, module geometries and reconfiguration algorithms are well catalogued [**?**, **?**], and platforms exist that dock autonomously and reconfigure on demand. What remains open is the decision, not the motion. Almost all of this work is *shape-oriented*: the target configuration is supplied from outside the system, and the robot computes how to reach it. On a planetary surface there is nobody outside the system to supply it. Round-trip light time to Mars runs from roughly six minutes to over forty, so a rover facing an unexpected obstacle cannot ask; it must decide.

That inversion — from "given this shape, reconfigure" to "given this obstacle, choose a shape" — is the problem this project addresses. It requires three things the literature has not

yet combined: perception of the obstacle from the robot's own sensors, inference of the capability the obstacle demands, and selection of a configuration that supplies it, all with computation carried on board. The nearest existing system closes a version of this loop but depends on a dedicated sensor module and off-board computation [**?**], and the learning-based approaches to configuration selection are almost entirely confined to simulation, where contact dynamics and magnetic docking are precisely what fails to transfer.

This proposal sets out the design, construction and evaluation of an MSRR that closes that loop end-to-end on real hardware. Section **??** establishes the technology trends, the societal need and the resources available, and states the problem precisely. Section **??** fixes the objectives and the criteria they will be measured against. The remaining sections cover the literature reviewed, the concept and methodology, the project timeline, and the theoretical background the starting period requires.

2 Background and Problem Statement

Current technology trends, societal needs, and the available resources.

2.1 Current technology trends

Modular self-reconfigurable robotics (MSRR) has matured from an academic concept into a technology actively pursued for space and search-and-rescue missions. Lattice, chain and mobile architectures are well documented [**?**, **?**], and a recent survey of mechanisms and design catalogues the available connectors and module geometries [**?**]. Demonstrated platforms span precision-docked chains (SMORES and SMORES-EP [**?**, **?**]), freeform magnetic systems (FreeBOT [**?**]) and outdoor tracked swarms [**?**]. The 2026 *Science Robotics* review frames the field's direction as on-demand multifunctional application rather than further mechanism proliferation [**?**].

The hybrid-joint module — where a module's degrees of freedom serve interchangeably as driving wheels or as articulated bending joints — was established by the SMORES platform, whose cubic modules carry four degrees of freedom, two of which act either as differential-drive wheels on flat terrain or as rotational joints once docked [**?**]. Electropermanent magnetic docking, which holds without continuous power draw, belongs to the later SMORES-EP work [**?**], not to the original 2012 design. The paradigm has since moved out of the laboratory: a snail-inspired modular swarm using a hybrid connector has been shown collectively adapting to unstructured outdoor terrain [**?**], evidence that terrain-adaptive reconfiguration is an open problem rather than a solved decade-old one.

In parallel, modular reconfigurable hardware is being pursued for space because of its redundancy, compact launch volume, and adaptability to unknown terrain. This is not conceptual only: decentralised reinforcement learning is being developed for modular reconfigurable lunar robots [**?**], and goal-oriented path planning has been demonstrated for modular self-reconfigurable satellites [**?**].

Software and onboard sensing are the bottleneck, not mechanism design:

- Most demonstrations depend on external motion capture, overhead cameras, or off-board compute. Self-contained relative localisation has only recently been treated as a problem in its own right [**?**, **?**].
- Perception pipelines assume a fixed sensor pose; a reconfiguring body breaks that assumption.
- Learning-based reconfiguration is almost entirely simulation-only [**?**, **?**, **?**, **?**]. Contact dynamics and magnetic docking are precisely what fails to transfer.

2.2 Societal and industrial needs

Space and planetary surface operations. Reconfigurable modules reduce launch mass and volume relative to shipping several single-purpose robots, and provide redundancy, self-servicing and repair where human intervention is impractical [**?**, **?**]. A conventional wheeled rover crosses a flat plain easily but is stopped by a crevasse, a crater-rim ledge, or a confined lava-tube void — the terrain that shape change exists to defeat.

Search and rescue. Unstructured, non-mappable environments where morphology change substitutes for routing around obstacles [**?**, **?**]. Urban search and rescue remains the standard proving ground for adaptive all-terrain mobility.

Inspection of hazardous or inaccessible infrastructure. Small openings leading to large interior volumes — pipes, ducts, damaged structures — where shape change is the only way in.

Manufacturing. Reconfiguration to match the current product on a line, suited to small-batch production.

2.3 Problem statement

A space search mission is specified before launch; the environment it meets is not. The object of a search is known as a description rather than a location, so neither the route to it nor the obstacle that blocks that route can be enumerated in advance, and orbital survey resolves terrain far coarser than the features that actually stop a vehicle. Launch mass then forbids the obvious response: a specialised vehicle per contingency is exactly what the budget cannot carry, and with no resupply each such vehicle is a single point of failure for its own speciality.

An MSRR answers both from one property — the modules are identical. One homogeneous set covers capabilities that would otherwise need several purpose-built vehicles, and any module substitutes for any other, so a failed unit is drawn from the same pool that supplies the capability range [**?**]. Mass spent on redundancy is not mass withheld from capability. That holds only if the robot can choose its own shape in the field.

2.4 Available resources

Prior art to build on, not reinvent. Two decades of MSRR research supply reusable building blocks. On hardware: proven connector designs and complete module architectures (SMORES [**?**], SMORES-EP [**?**], FreeBOT [**?**]) with published strength, cycle-time and power figures. On software: goal-oriented reinforcement learning [**?**], hierarchical mechanochemical control [**?**], and library-driven mission planning [**?**] provide a validated starting point. Foundational literature is already collected: the Stoy et al. textbook [**?**] and the Yim et al. grand-challenges survey [**?**].

Project resources. Four members; 200,000 THB; FDM printing, machining, electromagnetic field lab. Team background: ROS 2, SLAM, computer vision.

2.5 Scope and limitations

- Negotiate obstacles from three defined classes using onboard perception and compute only. No movement scripts, no prior map, no external localisation.
- Prototype-scale terrestrial testbed as a planetary analogue. No claim of space-flight qualification.
- Fixed module count in hardware, the count itself being an output of actuator sizing rather than an input. Scaling behaviour beyond that count is addressed in simulation only.
- Single defined working environment, to be specified with dimensions, surface and lighting.

3 Objectives

3.1 Main objective

Design, build and evaluate a number of MSRR modules and their physics-validated digital twin, such that the system perceives an obstacle, selects a configuration from a learned policy, reconfigures, and traverses — using onboard sensing and compute exclusively.

3.2 Specific objectives

#	Objective
1	Build a number of mechanically homogeneous modules with reliable autonomous dock/undock via an electropermanent magnet connector; a subset of them populated with camera and onboard compute.
2	Physics twin validated against measured hardware parameters — torque, mass, friction, dock success rate.
3	Onboard obstacle classification and governing-parameter estimation from a single camera, with no prior map.
4	Learned policy mapping perceived obstacle state to target configuration, generalising to held-out parameter values.
5	Reconfigure and traverse; the full perceive, select, reconfigure, traverse loop closed end-to-end.

3.3 Obstacles and configurations

Three classes, parametrically varied so that configuration selection is non-trivial.

Class	**Analogue**	**Governing parameter**	**Training range**	**Held-out**
Gap	Rille or crevasse	Width	Set by the torque analysis in Section **??**	
Step / ledge	Crater rim	Height	Set by the torque analysis in Section **??**	
Narrow passage	Lava-tube skylight	Width	Set by the torque analysis in Section **??**	

Configurations. Single module, linear chain, arch (cantilever). A fourth non-planar configuration if roll authority is retained.

On the parameter ranges. Ranges are an output of actuator sizing, not an input. Fixing them earlier produces a specification that cannot be defended in review.

3.4 Success criteria

Criteria are tiered Must / Should / Stretch. Each states the condition under which it is measured, because a docking success rate quoted without its starting conditions cannot be compared against published figures — docking from a pre-aligned start and docking at the end of an autonomous navigation run are different measurements.

Tier	Criterion	Measured by
Must	All planned modules built and individually functional	Each module drives under command, actuates every DOF through its full range, and engages and releases its connectors on demand.
Must	Digital twin operational	The twin reproduces a hardware-measured drive trajectory and a docking approach within a stated tolerance; every planned configuration loads and remains stable under gravity.
Must	Connector reliability: dock and undock succeed in at least 18 of 20 trials	Trials start from a **pre-aligned pose**, with the moving module placed inside the capture envelope. This isolates the connector mechanism from navigation and localisation error.
Must	Policy success at least 80 % in simulation, including held-out randomisation parameters	Held-out set drawn from outside the training ranges of Section **??**; reported per configuration.
Must	Onboard camera and compute verified	Perception pipeline runs onboard at 4 Hz.
Should	Docking after autonomous navigation succeeds in at least 14 of 20 trials	Full sequence from a commanded start pose: navigate, adjust pose, approach, engage. The lower target reflects the additional failure modes navigation and localisation introduce.
Should	Full autonomous loop on one obstacle class in hardware, at least 7 of 10 trials	Perceive, select configuration, reconfigure, traverse — no operator intervention between start and traversal.
Should	Learned policy outperforms the scripted gait on at least one contact-rich configuration	Both controllers evaluated over the same trial count on the same obstacle; the policy must win on traversal success *and* traversal time.
Stretch	Two obstacle classes in hardware	As above, for a second class requiring a different configuration.

Tier	Criterion	Measured by
Stretch	Decentralised policy using local observation and neighbour messages	Policy runs per module with no shared state beyond messages exchanged with directly connected neighbours.
Stretch	Goal-directed search within the arena after traversal	The assembled robot locates a specified target beyond the obstacle.

Calibration of the hardware targets. These figures are set against the closest comparable end-to-end result on this class of hardware. Daudelin et al. [**?**] categorise 24 recorded failures of one SMORES-EP demonstration: 10 (41.7 %) hardware, 6 (25 %) perception, 4 (16.7 %) human error, 3 (12.5 %) navigation, 1 (4.2 %) network. Hardware reliability, not algorithm quality, was the largest single failure cause on a mature research platform. Note that this is a breakdown of failure causes, not a success rate — the paper does not report one, and the figures should not be quoted as a proportion of attempts. Two consequences are built into the criteria above. First, connector reliability is measured separately from the full loop, so a hardware limitation is visible as such rather than absorbed into an end-to-end number. Second, the post-navigation docking target sits deliberately below the pre-aligned target, since the reference system's experience indicates that the additional stages carry most of the risk.

Reporting convention. Criteria are stated as trial counts rather than percentages. At ten or twenty trials the confidence interval around a success rate is wide, and reporting "7 of 10" rather than "70 %" keeps the sample size visible to the reader.

4 Literature Survey and Review

4.1 Locomotion mechanism design

Reviewed SMORES and SMORES-EP for the four-DoF layout and the use of the connector disks as differential-drive wheels [**?**, **?**]; PolyBot for open-loop, table-driven chain gaits [**?**]; Sambot for a mobile module with a separately rotating docking face and a self-locking worm drive [**?**]; and CEBOT, SMC, Millibot and Swarm-bots as mobile systems that lose chain locomotion once docked [**?**].

4.2 Communication system (data transmission)

Reviewed SMORES (XBee) and SMORES-EP (Wi-Fi to a central computer, with pose from external motion capture) for wireless approaches [**?**, **?**]; PolyBot (CAN) and M-TRAN (electrical contacts) for buses routed through the connector [**?**, **?**]; CONRO and CKBot for per-face infrared [**?**, **?**]; Sambot for a two-medium scheme switched at docking [**?**]; and the EP-Face inductive UART channel [**?**].

4.3 Low-level control and sensing

Reviewed SMORES for the single-MCU architecture, motor drivers and potentiometer joint sensing [**?**]; Sambot for a master-and-slave controller arrangement with encoders, gyroscope and accelerometer [**?**]; PolyBot G3 for Hall-effect joint sensing [**?**]; and CONRO for distributed control running on an 8-bit microcontroller [**?**].

4.4 Reconfiguration planning

Reviewed Liu et al. [**?**] for the SMORES-EP self-assembly framework — root-module search, Kuhn–Munkres task assignment, and depth-wise parallel action groups — adopted here as the planner's backbone since it targets the same hardware; Liu and Yim [**?**] for the connection-graph notation used to describe a configuration (Face, Face2Con, Orientation); Binder et al. [**?**] and Binder [**?**] for prioritised multi-robot route planning with spatio-temporal vertex locking, collision-resolution strategies and precondition-based execution, named by Liu et al. as the planner for their navigation stage and adopted here; Daudelin et al. [**?**] for the only end-to-end demonstration of perception-driven reconfiguration on this hardware — environment classification, library-based configuration selection, autonomous reconfiguration and task execution — which supplies the system template, the localisation arrangement adopted for Phase A (RGB-D SLAM for the environment map, AprilTag fiducials read by a downward-facing camera for module pose during reconfiguration), and — in the authors' own stated limitation of "oversimplified environment classification" — the gap this project targets; Jing et al. [**?**, **?**] for the property-indexed design library that selection mechanism rests on, in which each entry pairs a configuration and behaviour with the environment properties it suits (stair height, tunnel height, gap width, ground roughness); and Liu et al. [**?**] for distributed reconfiguration planning, not adopted because its per-module autonomy is unnecessary at a fleet size of 4–6 and adds coordination overhead our centralised staging does not require.

4.5 Simulation and assembled locomotion

Reviewed Gazebo [**?**] and MuJoCo [**?**] as candidate simulators — the former for its ROS 2 integration and sensor models, the latter for its contact model and for equality constraints that can be toggled at runtime, which map directly onto docking and undocking; MJX, MuJoCo's accelerator-backed branch, for reinforcement-learning throughput; ROS 2 [**?**] as the integration layer holding the control loop; PPO [**?**] as the policy-gradient method for contact-rich gaits; Tobin et al. [**?**] and Peng et al. [**?**] for domain randomisation as the standard route to sim-to-real transfer, the latter for dynamics rather than visual randomisation; Whitman et al. [**?**] for modular-robot control policies conditioned on assembled topology, reviewed but not adopted because our configuration count is small enough to treat each assembled configuration as a distinct robot and train one policy per configuration, rather than one network generalising across topologies; PolyBot's open-loop table-driven gaits [**?**] together with the hand-authored behaviour library of Jing et al. [**?**, **?**], in which each configuration carries a manually designed behaviour, as the established state of practice for assembled-configuration locomotion and therefore the baseline against which learned policies are measured; and Daudelin et al. [**?**] for the fragility of open-loop behaviours under environmental variation, which the authors of that system identify

as one of its stated limitations and which motivates learning closed-loop policies for contact-rich configurations.

4.6 Positioning and alignment system

Reviewed UWB–INS-based relative positioning for robust relative localisation of unmanned ground vehicles, using UWB measurements with an error-state iterated extended Kalman filter (ES-IEKF) to estimate relative position [**?**]; Sambot for its two-stage infrared docking approach, using passive IR sensing for approach and active IR sensing for final docking alignment together with IMU feedback [**?**]; and SMMRob for modular self-reconfiguration and coordinated localisation among modules [**?**]. These approaches motivate a hierarchical positioning and alignment system for SMORES-EP, in which UWB provides coarse relative positioning over larger distances, while IR sensing and the IMU are used for short-range alignment and docking.

4.7 Docking interface

Before committing to a connector, we weighed nine designs across FreeBOT [**?**], CAST [**?**], the snail-inspired hybrid connector [**?**], SMORES-EP [**?**], Roombots [**?**], M3Express [**?**], the self-aligning EPM interface [**?**], TYCOON [**?**], and the UBot multi-sensory docking approach [**?**]. FreeBOT, the snail connector, and UBot were dropped early: FreeBOT's point-magnet attraction is too weak in tension to support a lifted cantilever, the snail connector is mechanically complex and impractical to implement within our design constraints, and UBot's staged vision/Hall-sensor approach solves a perception problem we are already solving ourselves rather than a connector problem.

CAST and the self-aligning EPM interface were set aside for the opposite reason. They are compliance-driven designs that reduce the precision our navigation controller must hit, but neither has been demonstrated on a self-mobile module with the wiring, sensing, and structural loads our task requires. M3Express and Roombots were dropped on power and load grounds respectively: M3Express's off-board control model conflicts with our onboard-perception requirement, and Roombots' motor-actuated latch is sized for furniture loads well beyond our torque budget.

This leaves TYCOON and SMORES-EP as the two strongest candidates. TYCOON is particularly promising for the lifting problem, as its complementary intermittent gear mechanism allows one module to reinforce a neighbouring joint's torque without requiring a second motor, potentially reducing the cantilever risk identified in chain configurations. However, SMORES-EP's genderless docking interface provides greater freedom of movement while enabling fast, low-power state changes, and its EP-Face has a published characterisation of holding force and acceptance area [**?**]. We therefore select SMORES-EP's electropermanent connector as the baseline, while treating TYCOON's torque-sharing mechanism as a possible future addition to the joint drivetrain rather than a replacement for the docking interface.

4.8 Task-oriented planning

The reconfiguration planning most relevant to this project is the SMORES-EP line of work from the ModLab group. Liu, Whitzer and Yim [**?**] present a distributed reconfiguration planner that transforms an initial configuration into a target configuration while minimising disconnect and reconnect operations. Liu, Lin, Kim and Yim [**?**] extend this into a parallel self-assembly framework using Kuhn–Munkres task assignment and depth-based grouping of docking actions. Both approaches assume that the target configuration is already given, making them suitable as the shape-oriented baseline for our reconfiguration system, which we intend to leave unchanged.

Daudelin et al. [**?**] instead address the configuration-selection problem by choosing from a library of pre-verified configurations based on task and environmental requirements, while treating the actual reconfiguration as a separate process. Other studies explore configuration selection through graph-based methods, including centre-configuration selection [**?**], maximum-similarity subchain matching [**?**], subconfiguration decomposition [**?**], and graph-isomorphism-based assignment [**?**]. Learning-based approaches have also been explored, including joint configuration and motor-primitive selection [**?**], multi-agent reinforcement learning for freeform reconfiguration [**?**], and selecting between existing reconfiguration algorithms [**?**].

Existing work shows that modular robot configurations are primarily represented as graphs or bounded sets of predefined configurations, rather than being generated directly from raw environmental data. The literature also suggests that using a limited set of pre-verified configurations provides a more practical starting point than unrestricted configuration search.

4.9 Built-in camera for perception

Reviewed SMMRob for onboard visual perception in compact self-assembling modular robots, demonstrating that visual sensing can enhance perception and support relative positioning and docking [**?**]; terrain traversability analysis using RGB-D vision and terrain probing to evaluate geometric, semantic and physical properties of unknown environments [**?**]; lightweight multi-zone time-of-flight depth sensing with an 8×8 sensor and low-computational-cost decision making for obstacle avoidance on resource-constrained robots [**?**]; real-time robot vision on low-performance computing hardware using lightweight visual processing methods [**?**]; and low-cost monocular vision for road detection and following in mobile robots [**?**]. These studies motivate a lightweight onboard perception system for SMORES-EP that combines a monocular camera for visual terrain perception with compact ToF sensing for local depth information. The resulting terrain and traversability information can then be used by the leader module to support transformation selection, rather than relying solely on conventional object detection.

4.10 Compute module

Reviewed SMORES-EP for onboard processing used alongside external motion capture [**?**]; CKBot for a dedicated camera module carried one per cluster with processing on the camera [**?**]; and the end-to-end task system for AprilTag localisation running on an external laptop [**?**].

5 Project Concept Development

Theoretical background, technical depth, innovation, and practical relevance.

The system is developed in four phases. Each phase keeps the mechanical platform of the one before it and moves one dependency from outside the robot to inside it, so that a failure at any stage is localised to the dependency just internalised rather than to the platform as a whole.

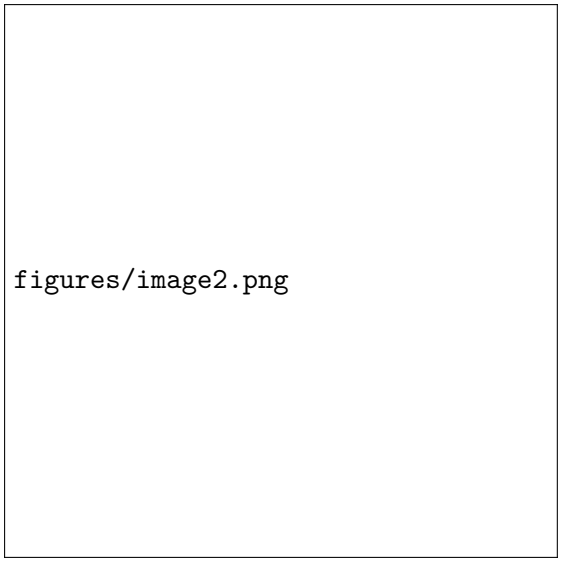
Phase	Definition
A	External perception and computation. Basic locomotion of the individual module with three-DoF onboard odometry and localisation, following the SMORES architecture, with no inter-module contact — locomotion only. Communication is a manually configured star topology to the commander. The system is shape-oriented at this stage. *Goal: reverse-engineer the SMORES platform.*
B	External perception and computation. The same module fitted with the SMORES-EP electropermanent connector, now with contact — chain, lattice and mobile reconfiguration. The system becomes task-oriented.
C	Internal perception, external computation. The same SMORES-EP module, with sensors mounted on the leader module and sensor data carried over a wired link to an external processing unit.
D	Internal perception and internal computation on the same SMORES-EP module — the full onboard target.

5.1 Phase A — Locomotion, sensing and communication

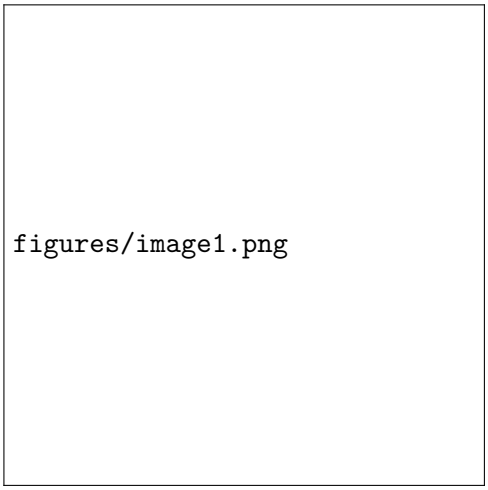
Locomotion mechanism design

Building a hybrid modular robot that combines chain, lattice and mobile reconfiguration gives two distinct problems. First, the module must drive itself on the ground to a docking site or to another node. Second, once assembled it must act as a joint. Speed drives the first, torque drives the second, and both run through the same gear train. SMORES is the only published platform that does all three reconfiguration styles in one module [**?**, **?**], so we will reverse-engineer and mimic that architecture first, then enhance the holding torque at the joints to allow more connected modules and a wider range of assembled actions.

The module inherits the SMORES-EP four-DoF layout: left, right and pan rotate continuously, and tilt has a limited angular range. The two side rotations double as wheels for differential drive, and the same two faces serve as docking faces. SMORES produces its four DoF with four



(a) Module layout and gear train.



(b) Degrees of freedom.

Figure 1: The SMORES four-DoF hybrid module architecture adopted as the Phase A baseline [**?**].

298:1 gear motors, each driving a 9-tooth pinion into a 48-tooth spur gear for a further 5.33:1 [**?**]. Left and right each have their own motor, so they can be commanded independently for steering. Pan and tilt come from the other two motors through a 48-tooth crown gear coupled to both inner spur gears: counter-rotating them pans, co-rotating them tilts. Only the tilt case combines both motors' torque, which makes tilt the strongest DoF — useful in the chain configuration, where tilt carries the cantilever load.

Low-level control and sensing

For low-level control, a microcontroller on every module handles joint control, sensor sampling and the data link. The original SMORES ran five motors, including the latch motor, from a single mbed LPC1768 (Cortex-M3, 96 MHz) [**?**]. We plan to use either an STM32 or an ESP32 family part, each for a different reason. The STM32 is built for actuation control and has CAN support on chip, which keeps a wired inter-module protocol open as a future option. The ESP32 has Wi-Fi and Bluetooth built in, so no additional wireless module is needed. The two differ in core architecture — the STM32G4 is an ARM Cortex-M4, while the ESP32 family uses Xtensa or RISC-V cores — so the choice also fixes the toolchain.

For joint position the original SMORES used continuous-rotation potentiometers with 5° resolution [**?**]; we will use a Hall-effect or magnetic encoder instead, to reduce the error carried into joint odometry. Each joint runs a closed-loop position controller on the module's own MCU from encoder feedback, driving the motors through dual H-bridge drivers — SMORES used three TB6612FNG parts for five motors [**?**]. Current sensing gives torque feedback and detects stall or contact, which is also how the module confirms it has reached a docking face. An IMU is added for attitude and orientation relative to gravity; SMORES lists this as future work, and Sambot already carries one (ADXRS613 gyroscope, ADXL335 accelerometer) [**?**].

The module also needs internal room for the EP contact system, the IR and UWB alignment sensors, and the connection to the compute module carrying the perception system.

Communication system (data transmission)

The link must work in two conditions — when modules are on their own, and when they are connected as one structure — and it should not change medium at docking.

The original SMORES used an XBee radio so that "the docking connections need only worry about mechanical attachment" [**?**]. SMORES-EP moved to Wi-Fi with an onboard processor on every module at about 40 Hz, but still to a central computer, and takes module pose from an external VICON system at 100 Hz [**?**]. We plan to remove both external dependencies.

We will use Wi-Fi. For the ESP32 it is already on chip, so no extra radio is needed. A radio keeps communication independent of the docking mechanism — the same reason SMORES used XBee. PolyBot and M-TRAN instead route a bus through the connector [**?**, **?**], which adds a contact reliability failure at every joint and constrains the connector design. One radio also covers both conditions with no handover, unlike Sambot, which switches from ZigBee to CAN once docked [**?**].

SMORES-EP reports a second option we may use later. Connected EP-faces can exchange data through the magnetic coupling of the mated EP-magnets, and the paper states the faces are capable of UART serial communication [**?**]. This is contactless, so it avoids the contact reliability problem of a wired bus, but the paper only reports the capability and does not characterise it, so there is no measured rate or error figure to design against. It is also unlikely to be a high-bandwidth channel, since the magnets are driven by 3 ms pulses. Its real value is that the channel is face-local, so a message identifies which face it arrived on. We treat it as a possible addition for topology discovery, not as a replacement for the radio, so the baseline never depends on a medium change at docking. For the STM32G4, its built-in CAN stays available as a wired option on the same terms, once the EP contact system can carry data.

The link carries the data needed to run the cluster: joint angles from each DoF, IMU attitude, connector state on each face, battery state, module ID and sequence number, topology updates, and the commands and configuration decisions sent by the coordinator.

Reconfiguration planning

Reconfiguration planning takes a target configuration from the perception subsystem together with a set of modules at arbitrary poses, and returns which module goes where, in what order, and how each one drives there and connects. It runs in three stages. *Task assignment* maps physical modules to target positions so as to minimise total travel distance, using the Kuhn–Munkres algorithm over a cost matrix of relative-pose distances. *Parallel assembly sequencing* walks the target tree breadth-first from its root and groups all actions at a given depth into a single group executed concurrently, reducing the schedule from $O(n)$ sequential docking operations to $O(depth)$ groups — the central time saving, since docking time, not computation, is the binding cost: reconfiguration in the reference system costs roughly one minute per module movement [**?**], against a planning cost measured in milliseconds. *Docking* carries each module to a waypoint one segment short of its parent, aligns the two connectors, and drives straight in until the electropermanent connectors engage.

Navigation layer. For the navigation stage Liu et al. name, but do not integrate, the prioritised route planner of Binder et al. [**?**]; we adopt it, and Section **??** works out what adopting it entails.

Its routing table attaches a precondition of the form "robot x has passed step n" to each segment, so a module waits on a peer's published progress rather than on wall-clock timing. This is what makes the parallel action groups of the previous stage executable rather than merely schedulable. The planner is incomplete, in that prioritised planning can fail on solvable instances; an action group that cannot be routed after rescheduling therefore degrades to sequential execution, which costs time but leaves the assembly sequence valid.

Lateral docking retained. A nonholonomic base cannot translate sideways into a LEFT/RIGHT mating pose. Liu et al. resolve this with a helper module that docks to the child's TOP face, lifts it clear of the ground, manoeuvres it into position and sets it down; we implement the same manoeuvre rather than excluding the connection type. The cost is one module of the 4–6 budget committed as helper for the duration of any lateral action, and an actuation requirement — a module able to raise a peer's weight without tipping — that feeds back into the mechanical design of Section **??**.

We accept that cost because excluding lateral docking is far more expensive than it first appears. Each module carries exactly one TOP and one BOTTOM connector, so restricting edges to TOP–BOTTOM pairs caps every module's degree at two, and the reachable topologies collapse to linear chains — no branching, no side-by-side widening. Measured against the configurations demonstrated in this lineage, only the chain-type designs would survive: Car, Scorpion, DoubleDriver and SwerveLifter all require LEFT/RIGHT edges [**?**, **?**]. The parallelism argument degrades with them, since a chain rooted at its centre offers only two actions per depth level. Retaining lateral docking is what keeps branching trees reachable, and with them both the published configuration library and the $O(depth)$ schedule that motivates adopting this framework at all.

One configuration remains out of reach for an unrelated reason: RollingLoop is a closed cycle, and the framework assembles trees only (Section **??**). The sequencer must reject any target configuration containing a cycle and return it to configuration selection rather than emit an infeasible plan.

Module localisation. Task assignment, route planning and path following all require module poses expressed in a common frame. For Phase A we adopt the scheme demonstrated by Daudelin et al. [**?**]: an RGB-D camera builds the environment map via RTAB-MAP SLAM, and AprilTag fiducials mounted on each module are read by a downward-facing camera covering the reconfiguration area to supply module pose during navigation and docking. This is external perception, consistent with the Phase A scope; the transition to onboard odometry and localisation belongs to Phase D.

Simulation and assembled locomotion

Simulation tooling is selected per phase, not once for the project. No single simulator serves planner validation, driving, contact-rich docking and RL training equally well, and committing early to one forces compromises in the others.

Need	**Tool**	**Why**
Planner validation (assignment, depth scheduling)	matplotlib / plain Python	2D kinematics only, no physics required; validated against the published task-assignment mapping in Liu et al. [**?**].
Phase A driving, no contact	Gazebo Harmonic + ROS 2	Matches the existing stack; contact fidelity not yet required.
Docking and assembled locomotion	MuJoCo	Weld equality constraints toggled at runtime give docking and undocking directly; strongest contact model of the candidates.
RL training throughput	MJX, without ROS 2 in the loop	Millions of environment steps through a message bus would dominate wall-clock time for no benefit.
Perception simulation (RGB-D, AprilTag fiducials)	Gazebo Harmonic	Gazebo carries camera and depth sensor models, so the Phase A localisation scheme above — RGB-D SLAM plus fiducial tracking, after Daudelin et al. [**?**] — can be exercised there. MuJoCo does not model these sensors, so perception-in-the-loop tests stay on the Gazebo side.

Integration. ROS 2 holds everything in the control loop — planner, docking controller, mode supervisor and inter-module messaging. Training runs outside it, against MJX directly, and trained policies are wrapped as ROS 2 nodes afterwards; a thin bridge connects MuJoCo state to ROS 2 topics where a policy must run in the loop. Newton / MuJoCo Warp is the throughput upgrade path should MJX become the bottleneck, and models carry over, so it is not where we begin.

Each assembled configuration is modelled and trained as a single robot. Once docked, a configuration is treated as one fixed-morphology machine with its own model, its own policy and its own gait, rather than as n modules under a shared policy that generalises across topologies. This makes assembled locomotion an ordinary single-robot reinforcement-learning problem, solvable with off-the-shelf PPO and standard tooling, and it is affordable at our scale. It also mirrors how the established design libraries are organised: Jing et al. [**?**, **?**] pair each configuration with its own hand-authored behaviour, so a per-configuration policy is the learned

analogue of an existing structure rather than an arbitrary simplification. The cost is that nothing transfers between configurations: a new morphology means a new model and a new training run from scratch. We accept this because the configuration set is small and fixed in advance; a topology-conditioned policy of the kind Whitman et al. [**?**] describe is the right choice at a larger configuration count, and is recorded as the scaling path rather than adopted now.

Two model forms, generated from one source. Simulating reconfiguration requires a scene of separate module bodies with weld constraints toggled at runtime; training a locomotion policy is faster and more stable on a fixed kinematic tree representing the assembled configuration. We keep both, and generate the per-configuration model programmatically from the connection graph defined in Section **??** — the same representation the planner uses — so the two cannot drift apart as the module design changes. The interfaces in the generated model are compliant joints carrying the measured connector stiffness, not rigid fixed joints: training on a rigid tree would train the policy on a stiffer robot than the hardware is, which is the same error the randomisation strategy below exists to avoid.

Control of assembled locomotion is split on whether a usable model exists. A configuration whose wheels remain grounded and whose wheel axes stay parallel after assembly is a rigid body with wheels, and is controlled classically as a differential drive; no learning is warranted. Configurations that traverse by contact — stair climbing, gap bridging — are trained with PPO [**?**]. The classical case is narrower than "any configuration with wheels": a vertical stack lifts the upper modules' wheels clear of the ground, so each configuration is classified explicitly rather than by default.

Every configuration additionally receives a scripted gait, which serves twice: as the fallback if a learned policy fails to transfer, and as the quantitative baseline the policy is measured against. This baseline is not a strawman — hand-authored open-loop gaits are the state of practice in this lineage, from PolyBot's gait tables [**?**] to the manually designed behaviours of the SMORES-EP design library [**?**, **?**] — so beating it is a result about the method rather than about our tuning effort. A finding of the form "traversal success rises from 60 % to 85 % against the hand-tuned gait" is defensible; "our policy works" is not. With lateral docking implemented, the reachable set is every tree topology within the module budget, so the configuration set can be drawn from the published library rather than restricted to chains: of an expected five configurations we anticipate two requiring learned policies, the contact-rich ones.

The scripted fallback also answers where these systems actually fail. Daudelin et al. [**?**] report that of 24 recorded failures of one demonstration, 41.7 % were hardware and 25 % perception — not planning. A controller that degrades to a known-good open-loop gait when a policy or a sensor misbehaves is therefore worth more than a marginally better policy.

Transfer is governed by the post-docking state distribution. A policy for an assembled configuration inherits whatever pose error reconfiguration leaves behind, so training randomises precisely that: per-interface position and orientation offset, joint angles at handover, approach angle to the obstacle, actuator backlash, and connector compliance [**?**, **?**]. This is where the locomotion subsystem couples to the reconfiguration subsystem — the randomisation ranges are not free parameters but must be matched to the residual alignment error measured in the docking trials of Section **??**. Note also that the docking mechanism is modelled as a weld equality constraint

toggled at runtime, which supplies the topology change; the compliance is then obtained by softening that constraint's solver parameters, not by treating the connection as rigid.

Scope. One configuration and one obstacle, carried through to hardware, before a second is added. Gap crossing is the first target: it transfers more readily than stair climbing, which demonstrates better but carries the higher transfer risk and is therefore scheduled as the stretch objective.

First milestone. Two module bodies in MuJoCo — weld at one face, unweld, re-weld at a different face. This is the smallest test of the entire simulation architecture: if runtime topology change is clean, the approach holds; if it is not, that is discovered before the detailed module model is built rather than after.

Positioning and alignment system

Concept and theoretical background. Building on the self-assembly architecture of SMMRob [**?**] and the IR-based docking principles of Sambot [**?**], this project proposes a cascaded positioning and alignment system for modular self-reconfigurable robots. The system divides autonomous docking into different operating ranges, using UWB and the IMU for macro-scale relative positioning and IR sensing for local alignment and final docking. Unlike camera-based approaches that require visual sensing on individual modules, the proposed architecture uses a designated leader module for positioning, while follower modules do not require onboard cameras.

Macro-scale relative positioning: UWB + IMU. During the initial rendezvous, UWB ranging is combined with high-frequency IMU measurements to estimate the relative position between leader and follower. An error-state invariant extended Kalman filter (IEKF) fuses these measurements and improves robustness against noise and UWB outliers, including non-line-of-sight errors [**?**]. This provides anchor-free relative positioning without requiring external localisation infrastructure.

Micro-alignment and docking: IR-based alignment. When the follower enters the local docking region, the system transitions from UWB-based positioning to IR-based local alignment, inspired by the IR-based autonomous docking mechanism demonstrated in Sambot [**?**].

The proposed IR system uses two functions:

- **Approaching IR:** provides local directional information to guide the follower toward the docking interface.
- **Docking IR:** provides close-range alignment feedback to reduce lateral and angular misalignment before mechanical connection.

The IMU is additionally used to provide attitude information during the final alignment process.

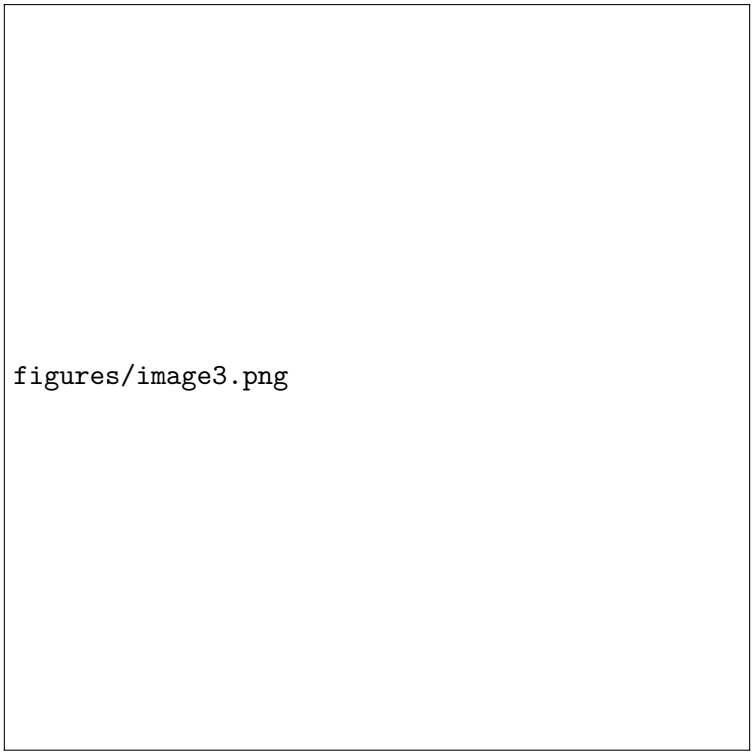


Figure 2: Two-stage infrared docking scheme: approaching IR for local direction, docking IR for close-range alignment, with IMU feedback supplying attitude [**?**].

5.2 Phase B — Docking and task-oriented reconfiguration

Docking interface

This docking interface is developed as the next stage after Phase A, which focuses on basic locomotion using the SMORES-based module design. The docking system builds directly on the Phase A module rather than introducing a completely new mechanical platform.

The EP-Face connector used in SMORES-EP is based on an electropermanent magnet (EPM), a type of magnet introduced for modular robotics by Knaian [**?**] and later adapted for the SMORES-EP platform by Tosun et al. [**?**]. Unlike a conventional electromagnet, an EPM can switch between an attractive and a non-attractive state using a short electrical pulse, without requiring continuous power to maintain its state.

This makes the EP-Face particularly suitable for our modular self-reconfigurable robot. In its docked state, the connector can provide a strong holding force while consuming almost no standby power. SMORES-EP's EP-Face has been reported to provide up to approximately 90 N of holding force, while switching requires only short current pulses [**?**, **?**]. This is useful for our design because modules may need to remain connected for long periods, while communication between modules already consumes significant power.

Unlike the locomotion-only Phase A design, where the module does not need to interact physically with other modules, the new design requires docking from multiple directions. EP-Faces will therefore be placed on all four sides of the module. Having identical docking interfaces on all four sides supports different robot configurations and gives the reconfiguration system greater freedom when forming chains, larger structures, or other shapes.

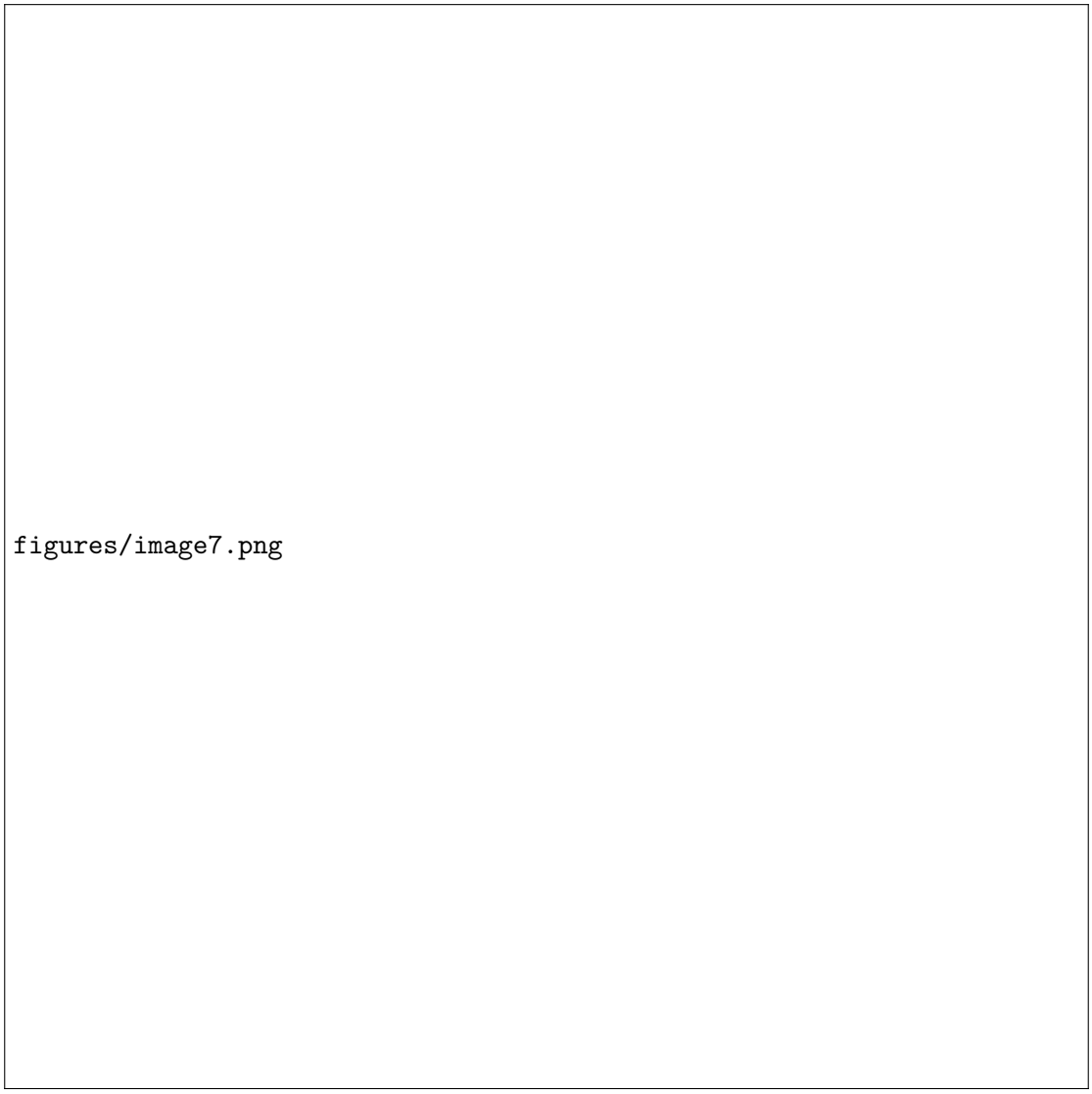


Figure 3: Electropermanent magnet operating principle and the EP-Face connector layout [**?**, **?**].

Another important advantage is the EP-Face's tolerance to alignment errors. The EP-Face uses multiple magnets arranged around the connector, allowing two faces to connect across a range of relative positions and rotations. Reported tolerances are approximately 4 mm of separation and 7 mm of lateral misalignment [**?**]. This reduces the precision required from the robot's movement and alignment system during docking, simplifying the control system. This is particularly useful when transitioning from Phase A's basic locomotion to physical interaction, as the locomotion system does not need to provide extremely precise positioning before a connection can be made.

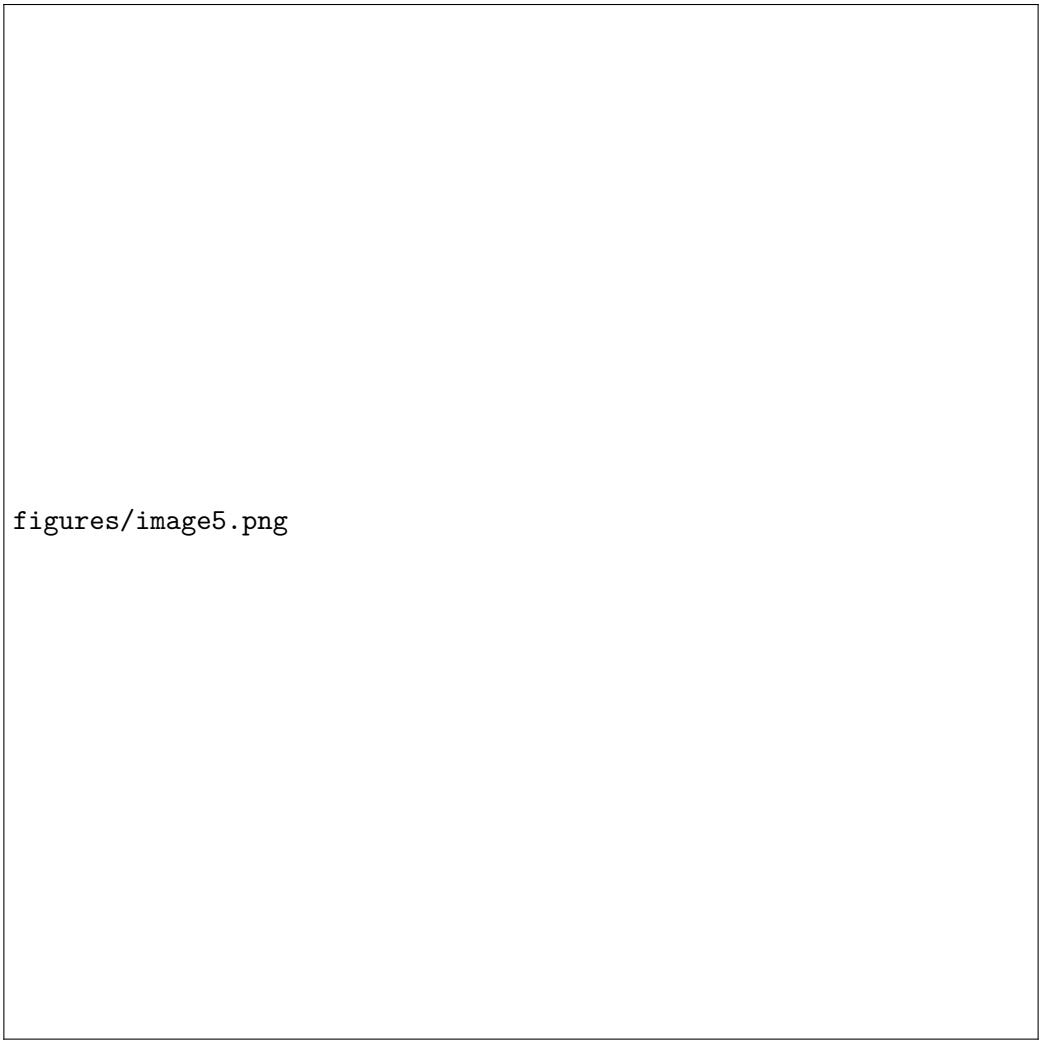


Figure 4: EP-Face connector geometry and its area of acceptance: the genderless magnet ring permits connection across a range of relative positions and rotations [**?**, **?**].

The connector is also genderless, meaning any EP-Face can connect to another EP-Face without requiring separate male and female interfaces. This is especially important because any side may act as either the connecting or the receiving side, depending on the desired configuration. The distributed magnetic arrangement also provides better resistance to forces and moments than a single central magnet, which is relevant to our concern about connections supporting cantilevered chains of modules.

Tosun et al. [**?**] also demonstrated that the EP-Face can support communication between connected modules. Although we do not rely on this feature in the baseline design, it could later

be used as a secondary method for confirming which modules are physically connected to each other.

The EP-Face is selected as our baseline docking mechanism because it combines low power consumption, strong holding force, alignment tolerance, genderless connection, and potential face-to-face communication. Placing the interfaces on all four sides extends the Phase A SMORES-based locomotion module into a physically interactive modular robot while preserving its basic mechanical concept.

Task-oriented planning

Phase B builds on the existing SMORES-EP reconfiguration pipeline. Once a target configuration is selected, the existing root selection, task assignment, and parallel docking process can proceed as described by Liu et al. [**?**]. The main addition in Phase B is therefore a new layer that determines which configuration should be used based on the task, rather than relying on a fixed shape.

The input to this layer can come in two forms. The first is a structured task requirement, such as the required reach, gap width, or payload capacity. The second is raw environmental information, such as a depth image or height map, from which these task requirements would first need to be estimated. Since selecting an incorrect configuration can be difficult and costly to test on real hardware, the system should be developed in stages rather than starting with a complex learning model.

The first stage may use a rule-based approach with a small library of predefined configurations. Each configuration will have a known capability range, such as maximum reach, gap-crossing ability, or load capacity. The planner will compare the task requirements with these capabilities and select the smallest configuration that can complete the task. This follows the discrete-library approach used by Daudelin et al. [**?**] while avoiding the need for training data. It is also easier to understand and verify before being used on the physical robot.

The second stage can replace the fixed library with a graph neural network (GNN) if the range of supported tasks becomes too large for a rule-based system. Since the robot configuration is already represented as a graph, a GNN can work directly with this representation instead of first converting the configuration into an image or other spatial representation. The same structural constraints used by the existing reconfiguration pipeline, such as tree topology and orientation constraints, can therefore be maintained [**?**].

A 3D CNN will be considered as a later stage if raw environmental sensor data becomes the main input. In this case, the CNN would be used primarily for perception, converting the sensed environment into task requirements such as required reach or gap width. It would not directly generate the final configuration. This keeps the configuration-selection process separate from environmental perception and allows the same selection system to be used regardless of where the task requirements originate.

The rule-based configuration library is selected as the baseline because it requires no training data, is easy to verify, and provides a simple starting point for testing on real hardware. The GNN approach is planned as a later extension because it works directly with the graph structure already used by the reconfiguration system. A 3D CNN is reserved for a future stage where raw

environmental perception becomes necessary, allowing it to act as a perception front end rather than replacing the existing configuration-selection process.

5.3 Phase C — Onboard perception with external computation

Concept and theoretical background

For modular self-reconfigurable robots such as SMORES-EP, autonomous exploration in unstructured environments requires onboard perception to support navigation and morphological adaptation. Conventional RGB-D cameras can provide rich depth information but may introduce additional size, weight, power and computational requirements. This project therefore proposes a low-SWaP perception system combining a compact monocular RGB camera with an 8×8 multi-zone time-of-flight (ToF) sensor. During the prototype stage, the sensors are mounted on the leader module, while sensor data are transmitted over a wired link to an external processing unit for real-time processing.

Terrain geometry perception using multi-zone ToF

An 8×8 multi-zone ToF sensor, such as the VL53L5CX, is used to provide a compact depth representation of the local environment. Müller et al. [**?**] demonstrated that a 64-pixel multi-zone ToF sensor can provide lightweight depth-based perception for autonomous navigation on a resource-constrained robotic platform, using a decision-tree-based approach with low computational and power requirements.

For the proposed MSRR, the ToF measurements are processed to estimate local geometric characteristics such as obstacle distance, height discontinuities, slope, and surface variation. These features provide a compact representation of the surrounding terrain without requiring dense point-cloud processing.

Monocular vision for supplementary terrain perception

A compact monocular RGB camera is used as a supplementary sensor to provide visual information about the environment. The camera is not intended for complex object bounding-box detection (e.g. YOLO) at this stage. Instead, a lightweight vision model will extract broad visual textures and semantic classifications of the surface — differentiating, for example, between a smooth floor, rough gravel, and vegetation. This visual texture information is then combined with ToF measurements to improve the overall perception of the local terrain and of traversable regions. This approach is motivated by previous terrain-perception research demonstrating that fusing visual semantics with geometric information significantly enhances the estimation of terrain properties and the construction of reliable traversability maps for robot navigation [**?**].

5.4 Phase D — Fully onboard perception and computation

Compute module

The compute module is added in the last phase, once locomotion, docking and communication are working. It is a separate board from the joint-control MCU and carries the camera and the perception software, so that the MCU keeps its real-time work while the compute module handles situation classification and configuration selection. We plan to fit it only to the module acting as coordinator, rather than to every module, following CKBot, which uses one dedicated camera module per cluster [**?**]; a camera on every module would cost power, volume and money we do not need at this scale. Candidates are the Raspberry Pi CM5 ($55 \times 40 \times 5\,\text{mm}$) and the STM32N6, which has an integrated NPU for lower power. The interface to the module MCU carries decisions and setpoints only.

6 Project Planning and Timeline

The project runs across two semesters. Phases A and B — locomotion with external perception, then docking and task-oriented reconfiguration — are delivered in Capstone Design Project I. Phases C and D, which move perception and then computation onto the module itself, are delivered in Capstone Design Project II. The split is deliberate: everything in the first semester can be demonstrated with off-board computation, so a delay in the compute hardware cannot stall the mechanical and docking work that the second semester depends on.

Date	Deliverable	Project state required

6.1 Milestones

Dates for the first semester are those set by the course guideline. Second semester dates are indicative and will be aligned to the Capstone Design Project II calendar when it is issued.

Date	Deliverable	Project state required
28 Aug 2026	Proposal submission	Concept, objectives and literature review fixed.
25 Sep 2026	First progress report	Phase A module designed; one module driving in simulation; connector selected and specified.
23 Oct 2026	Second progress report	First physical module built and driving; Phase A closed.
18 Nov 2026	Draft final report	Sub-system revision closed; locomotion and simple assembly demonstrated.
22 Nov–3 Dec 2026	Final presentation	Phase A module complete; Phase B build under way.
10 Dec 2026	Final report (Project I)	Phases A and B documented and evaluated.
Jan–Feb 2027	Phase C build	Camera and ToF mounted on leader module, wired to external compute.
Mar 2027	Phase C evaluation	Obstacle classification from onboard sensors, no prior map.
Apr 2027	Phase D integration	Compute module fitted; perception and selection running on board.
May 2027	Final evaluation	Full perceive–select–reconfigure–traverse loop closed on hardware.

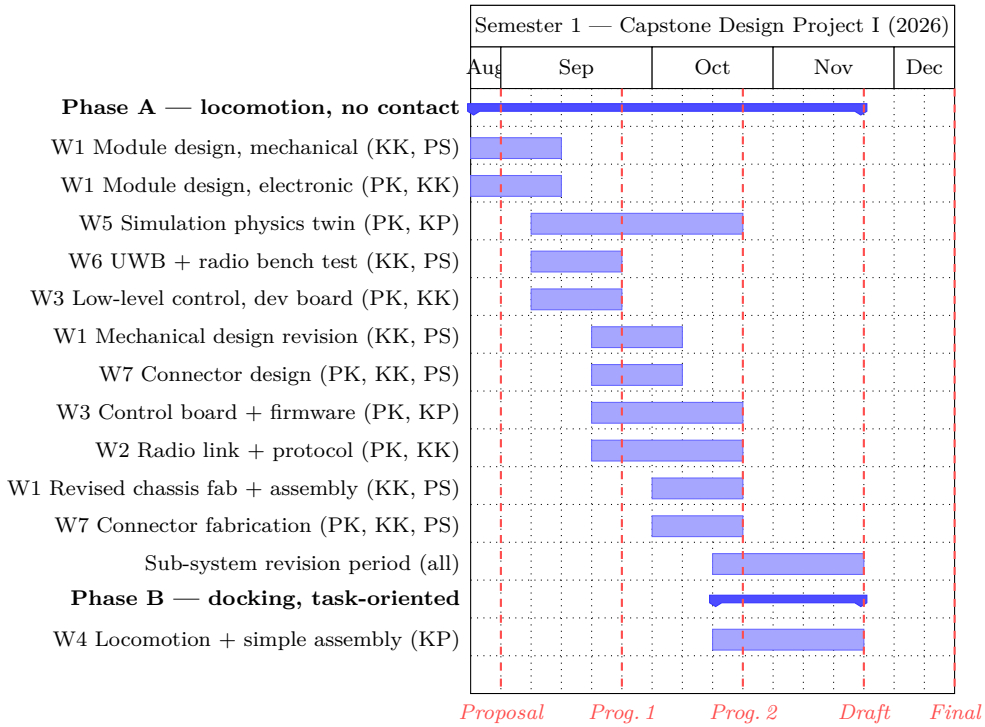
6.2 Workstreams and ownership

Ten workstreams run in parallel, each owned by one member who is responsible for its design decisions, its section of the report, and its verification. Ownership does not mean sole execution — build and test weeks are shared — but it fixes who answers for each subsystem.

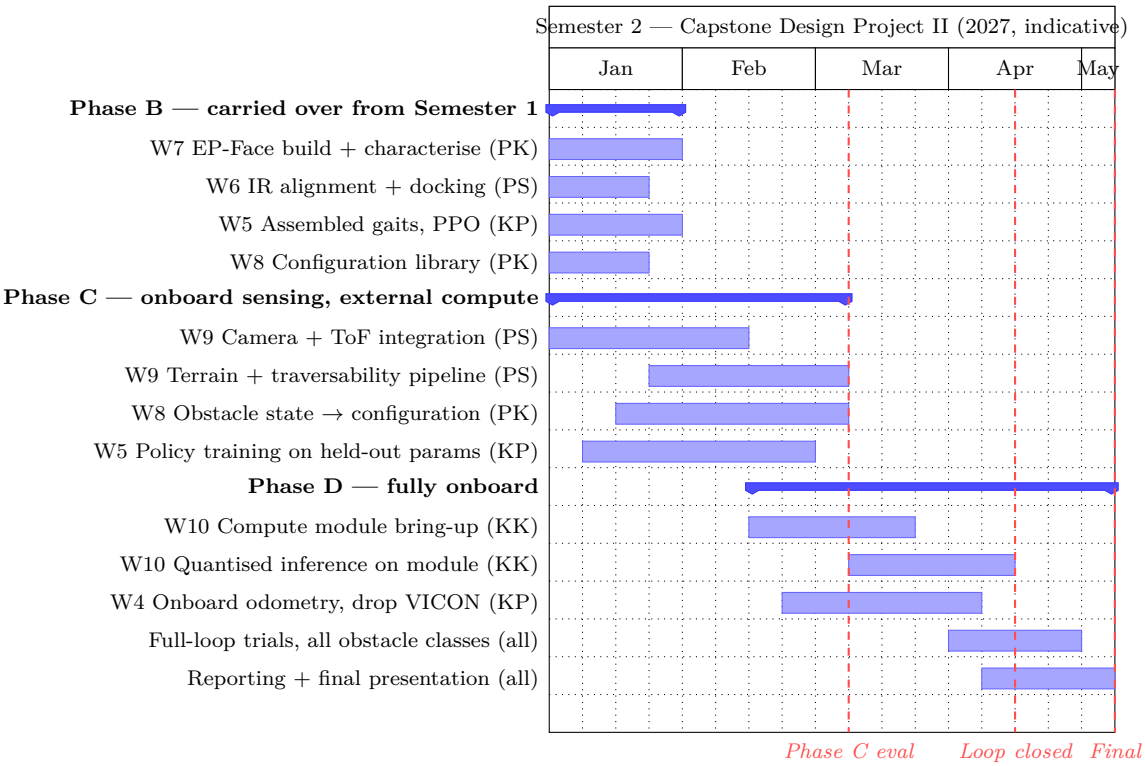
#	Workstream	Owner	Phase of first delivery
W1	Locomotion mechanism design	KK	A — module geometry and drivetrain
W2	Communication (data transmission)	KK	A — inter-module link
W3	Low-level control and sensing	KK	A — joint control, IMU, encoders
W4	Reconfiguration planning	KP	B — assignment and sequencing
W5	Simulation and assembled locomotion	KP	A — twin; B — assembled gaits
W6	Positioning and alignment	PS	A — UWB/IMU; B — IR docking
W7	Docking interface	PK	B — EP-Face connector
W8	Task-oriented planning	PK	B — configuration selection
W9	Built-in camera for perception	PS	C — camera and ToF pipeline
W10	Compute module	KK	D — onboard inference

Initials. KK — Krin Kaweewongsonthorn; KP — Kittivut Pipattanachaiyapong; PK — Pakapol Kijsupanont; PS — Phattharamon Sukvivatn. The schedule below labels each bar with the members actually executing it, which for most tasks is a pair; the owner above remains the single member accountable for the workstream.

6.3 Semester 1 schedule — Phases A and B



6.4 Semester 2 schedule — Phases B carry-over, C and D



7 Theory and Technical Backup

The theory below is what the starting period requires: the relations that size the hardware, bound what the electronics must achieve, and decide whether a design choice is feasible before anything is built. Each subsection states the governing relation and what it is used to fix.

7.1 Locomotion mechanism design

Driving on the ground. A module on its two side wheels is a differential drive. Body speed and yaw rate follow from the wheel angular velocities ω_L, ω_R and the wheel radius r over track width b:

$$v = \frac{r}{2}\left(\omega_R + \omega_L\right), \qquad \dot{\psi} = \frac{r}{b}\left(\omega_R - \omega_L\right). \tag{1}$$

Setting $\omega_R = -\omega_L$ gives $v = 0$ with $\dot{\psi} \neq 0$: the zero-radius turn used to correct heading immediately before docking, when the module must rotate without translating out of the connector's capture envelope.

Joint torque sets the module count. Once docked, the same degrees of freedom act as joints, and the binding case is the cantilever. For a chain of n modules of mass m at pitch L held horizontally, the root joint carries the summed moment of every module outboard of it:

$$\tau_{\text{root}} = mgL\,\frac{n(n+1)}{2}. \tag{2}$$

The quadratic growth in n is the reason module count cannot be chosen independently of actuator selection. This is compared against the delivered joint torque

$$\tau_{\text{out}} = \tau_{\text{motor}}\, N\, \eta, \tag{3}$$

for gear ratio N and drivetrain efficiency η, and the largest n satisfying $\tau_{\text{out}} \geq \tau_{\text{root}}$ is the characteristic strength of the design. Because the tilt degree of freedom combines both motors' torque through the crown gear (Section **??**), tilt is the axis this analysis is applied to.

Pan and tilt are not independently actuated. Both come from the same two motors through the 48-tooth crown gear, which makes the mechanism a differential gear train: counter-rotating the two inner spur gears pans the crown gear about DoF 3, co-rotating them tilts it about DoF 4 [**?**]. Writing θ_1, θ_2 for the two driven spur-gear angles, the joint angles are their difference and their sum,

$$\begin{bmatrix} \theta_{\text{pan}} \\ \theta_{\text{tilt}} \end{bmatrix} = \underbrace{\frac{1}{2}\begin{bmatrix} k_p & -k_p \\ k_t & k_t \end{bmatrix}}_{\mathbf{T}} \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix}, \tag{4}$$

with k_p, k_t the mesh ratios of the two paths through the crown gear. The fundamental train equation for a differential gives these ratios from the tooth counts [**?**].

The torque relation is the transpose of (**??**), which follows from virtual work and holds whatever the gear ratios turn out to be:

$$\boldsymbol{\tau}_\theta = \mathbf{T}^\top \boldsymbol{\tau}_q \qquad \Longleftrightarrow \qquad \boldsymbol{\tau}_q = \mathbf{T}^{-\top} \boldsymbol{\tau}_\theta. \tag{5}$$

Tilt therefore draws on the sum of the two motor torques and pan on their difference, which is why tilt is the stronger axis and why (**??**) is applied to it. SMORES measured 2.3 N·m in tilt against 1.4 N·m in pan at 9 V [**?**]; the ratio is 1.64 rather than 2 because the two paths through the crown gear have different mesh ratios and different losses. Reproducing those two numbers on our own module is the check that (**??**) has been implemented as intended.

Coupling also changes the dynamics, because motor inertia no longer reflects into one joint at a time. With $\mathbf{q} = \mathbf{T}\boldsymbol{\theta}$ and motor inertia matrix $\mathbf{I}_m$, the rigid-body model in joint coordinates becomes

$$\left(\mathbf{M}(\mathbf{q}) + \mathbf{T}^{-\top}\mathbf{I}_m\mathbf{T}^{-1}\right)\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \mathbf{T}^{-\top}\boldsymbol{\tau}_\theta, \tag{6}$$

the standard transmission-coupled form [**?**, **?**]. The off-diagonal terms of $\mathbf{T}^{-\top}\mathbf{I}_m\mathbf{T}^{-1}$ are the practical consequence: a pan command disturbs tilt and vice versa, so the two axes cannot be tuned as independent single-input loops. Either the controller is written in motor coordinates $\boldsymbol{\theta}$, where the axes decouple, or it compensates the coupling explicitly. We will take the first route and treat (**??**) as the interface between planner and joint controller.

Motor selection works from the DC torque–speed characteristic, linear between stall torque τ_s at zero speed and no-load speed ω_0 at zero torque, with peak mechanical power at the midpoint. The operating point must sit clear of stall for the cantilever case while still delivering the driving speed the traverse requires — the two demands that run through the same gear train, and the reason the gearing is a compromise rather than an optimum for either.

Traction, not torque, bounds gradeability. On an incline α the tractive force required is

$$F_{\mathrm{req}} = mg\,(\mu_{\mathrm{roll}}\cos\alpha + \sin\alpha)\,, \tag{7}$$

while the ground can supply no more than $\mu_s mg\cos\alpha$. Above a critical grade the wheels slip regardless of available torque, so the obstacle parameters of Section **??** must be checked against friction as well as against (**??**).

Assembled gaits. A chain traverses by propagating a travelling wave along its joints,

$$\theta_i(t) = A\sin(2\pi f t + \phi_i)\,, \qquad \phi_i = \frac{2\pi i}{n}, \tag{8}$$

with amplitude A and frequency f setting stride and speed. Configurations that lift modules clear of the ground are statically stable only while the projection of the centre of mass stays inside the support polygon, which is the constraint that limits how far an arch can reach before it topples.

7.2 Communication system (data transmission)

Load against capacity. Offered load is payload size times update rate times module count. Compared against nominal radio capacity this looks comfortable, but it is the wrong comparison: the traffic is many small frames, not few large ones, so the binding limit is channel airtime under CSMA/CA, where per-frame overhead and contention grow with the number of nodes rather than with payload size. The quantity to design against is therefore frames per second across the cluster, not bits per second.

Latency is fixed by the connector, not by preference. A module closing on a docking face at approach speed v_{approach} accumulates position error for as long as it takes a measurement to become an actuator response. For total loop latency t_{total} the residual error at contact is

$$e = v_{\mathrm{approach}}\,t_{\mathrm{total}}, \tag{9}$$

so for a connector with area of acceptance a the requirement is

$$t_{\mathrm{total}} \le \frac{a}{v_{\mathrm{approach}}}. \tag{10}$$

The EP-Face tolerances quoted in Section **??** therefore convert directly into a latency budget — and, read the other way, a slower approach buys latency at the cost of docking time.

Synchronisation for coordinated gaits. Joints on separate modules running a common travelling wave must share a time base. A clock offset Δt between two modules appears as a phase error

$$\Delta\phi = 2\pi f\,\Delta t, \tag{11}$$

so the tolerable offset scales inversely with gait frequency: faster gaits demand tighter synchronisation.

Topology is inferred, not measured. The cluster's connectivity graph is assembled from per-face neighbour detection, and each module's structural position is then obtained by propagating

coordinates outward from a root over that graph. No module measures its own position in the assembly; it derives it from who its neighbours are and on which faces.

7.3 Low-level control and sensing

Motor model. Both joint control and current-based torque feedback rest on the DC motor equations

$$V = IR + L\frac{dI}{dt} + K_e\omega, \qquad \tau = K_t I, \tag{12}$$

which state that torque is proportional to current — the basis for sensing load without a torque sensor.

Cascaded control. Each joint runs position over velocity over current. The loops are separated in bandwidth so that the inner loop settles well within the outer loop's sample period; without that separation the loops interact and the joint oscillates. PWM through an H-bridge sets average motor voltage by duty cycle, giving bidirectional control from a unipolar supply.

How the encoder recovers angle. An incremental encoder emits two square waves, A and B, in quadrature — B lagging A by a quarter cycle. Over one cycle the pair passes through four distinct states, $(A, B) = 00 \to 01 \to 11 \to 10$, and a decoder that counts every edge rather than every cycle therefore resolves four counts per line. For N_ℓ lines on the disc and gear ratio N between encoder and joint, joint resolution is

$$\Delta\theta = \frac{2\pi}{4N_\ell N} \text{ rad/count}, \tag{13}$$

and quantisation alone bounds position error at $\pm\Delta\theta/2$. Direction comes from the order of the transition, not from the count: the sequence above run backwards is $00 \to 10 \to 11 \to 01$, so a two-bit state machine comparing the present state with the previous one yields sign as well as magnitude [**?**]. Velocity is then obtained either by counting edges in a fixed window, which is accurate at speed and coarse near zero, or by timing the interval between edges, which behaves the other way round; the joint controller uses the second because docking happens slowly.

A magnetic encoder is absolute rather than incremental. The sensor reports two signals in spatial quadrature, and the shaft angle follows directly:

$$\phi = \operatorname{atan2}(V_{\sin}, V_{\cos})\,. \tag{14}$$

There is no index pulse to hunt for at power-up and no count to lose if a step is missed — both properties matter for a module that may be re-energised in an arbitrary pose after reconfiguration. This is the reason for moving away from the original SMORES continuous-rotation potentiometers, which resolved 5° [**?**].

Current sensing must be synchronous. Current is measured as a voltage across a shunt, $V = IR_{\text{shunt}}$, amplified before conversion. The shunt carries motor current only during part of the switching cycle, so an asynchronous sample returns a value that depends on where in the PWM period it happened to land. The ADC conversion must therefore be triggered at a fixed point in the PWM period — a timing requirement on the microcontroller, not merely a wiring choice, and one of the reasons for preferring a part with timer-triggered ADC.

Attitude from the IMU. With the module static, the accelerometer reads gravity alone and the two tilt angles follow from its three axes:

$$\phi_{\text{acc}} = \operatorname{atan2}(a_y,\, a_z)\,, \qquad \theta_{\text{acc}} = \operatorname{atan2}\Big(-a_x,\, \sqrt{a_y^2 + a_z^2}\Big)\,. \tag{15}$$

Roll and pitch are observable this way; yaw is not, because rotation about the gravity vector does not change the measurement. That is why heading has to come from the wheel odometry or from the positioning subsystem rather than from the IMU alone.

The estimate above is useless during motion, since the accelerometer cannot separate gravity from the module's own acceleration. Integrating the gyroscope instead,

$$\phi_{\text{gyro}}(k) = \phi(k-1) + (\omega_x(k) - b_x)\,\Delta t, \tag{16}$$

is clean over short intervals but accumulates: any residual bias error $\tilde{b}_x$ grows without bound as $\tilde{b}_x t$, and the random walk on top of it grows as $\sqrt{t}$. Neither sensor is sufficient alone, and their errors sit in opposite parts of the spectrum — the accelerometer is noisy but unbiased, the gyroscope smooth but drifting.

A complementary filter exploits exactly that split, low-passing the accelerometer and high-passing the integrated rate [**?**]:

$$\phi(k) = \alpha\,[\phi(k-1) + \omega_x(k)\,\Delta t] + (1-\alpha)\,\phi_{\text{acc}}(k), \qquad \alpha = \frac{\tau}{\tau + \Delta t}, \tag{17}$$

where the single tuning constant τ is the crossover time: below it the gyroscope is trusted, above it the accelerometer. A Kalman filter does the same job with the weighting derived from the noise covariances instead of chosen by hand, and additionally estimates the gyroscope bias b_x as a state rather than assuming it away. We will start with (**??**) because it has one parameter and can be verified on a bench tilt table, and move to the nonlinear complementary filter on $SO(3)$ [**?**] or a gradient- descent formulation [**?**] if the planar form proves insufficient once the module is climbing rather than driving.

7.4 Positioning and alignment system

Relative positioning and autonomous docking. An MSRR must determine the relative position and orientation of its modules before they can connect. Docking therefore divides into two tasks: *relative positioning*, which brings the modules into each other's vicinity, and *local alignment*, which reduces the remaining positional and angular error far enough for mechanical connection.

The proposed system uses different sensing for the two stages — UWB and IMU for longer-range relative positioning, infrared for short-range alignment. Each sensor is used across the range where its characteristics are best, which removes the need for any one technology to cover the whole docking process.

UWB-based relative positioning. Ultra-wideband sensing gives the relative distance between two modules from the propagation time of radio signals between their transceivers. In its simplest form the measured distance is

$$d_{\mathrm{UWB}} = \|\mathbf{p}_L - \mathbf{p}_F\| + v, \tag{18}$$

where $\mathbf{p}_L$ and $\mathbf{p}_F$ are the positions of the leader and follower modules and v is measurement error.

UWB suits the initial positioning stage because it supplies relative range without visual line-of-sight detection, which matters for a modular system operating where fixed external positioning infrastructure is unavailable. Its weakness is noise and signal obstruction: non-line-of-sight interference produces inaccurate or outlier measurements. That is what motivates fusion rather than relying on UWB alone [**?**].

IMU-based motion and orientation estimation. An inertial measurement unit supplies linear acceleration and angular velocity at high rate, from which the module's motion and orientation are estimated between UWB updates. UWB gives relative distance but not a continuous estimate of the full relative motion; the IMU propagates motion information across the gaps. Since inertial estimation drifts as error accumulates, combining the two exploits strengths that sit in opposite parts of the spectrum.

Fusion by invariant error-state Kalman filter. The two measurement streams are fused in an error-state invariant extended Kalman filter (IEKF), with a prediction step driven by the IMU and a measurement update from UWB:

$$\mathbf{x}_{k|k-1} = f(\mathbf{x}_{k-1},\, \mathbf{u}_k)\,, \tag{19}$$

$$\mathbf{z}_k = h(\mathbf{x}_k) + \mathbf{v}_k, \tag{20}$$

where $\mathbf{x}$ is the estimated state, $\mathbf{u}_k$ the inertial measurements, $\mathbf{z}_k$ the UWB observation and $\mathbf{v}_k$ the measurement noise. The invariant error-state formulation gives a mathematically suitable representation for relative pose estimation and improves robustness under measurement disturbance. State definition, propagation model, covariance equations and the full update procedure are developed in Section **??**; the use of UWB–IMU fusion for anchor-free relative positioning follows established relative-positioning work [**?**].

IR-based local alignment. As the follower approaches, the accuracy demanded rises sharply, because mechanical docking requires the interfaces to be aligned within the connector's capture envelope. Short-range infrared sensing covers this stage, supplying proximity and directional information from emitted or reflected signals. It is well suited to confirming that the follower is approaching the docking interface correctly and to providing local feedback during final alignment; earlier modular systems have demonstrated IR-guided autonomous docking [**?**]. The stage divides into two functions:

- *Approaching IR* — local directional information guiding the follower toward the docking interface.
- *Docking IR* — close-range feedback that removes the remaining lateral and angular misalignment immediately before connection.

The IMU supplies orientation throughout this stage, so the controller can resolve translational and rotational alignment together.

The cascaded architecture. Taken together the stages form one sequence rather than three independent systems:

Stage	Main sensors	Purpose
Macro-scale positioning	UWB + IMU	Estimate relative state and guide the follower toward the target.
Local approach	Approaching IR	Obtain local directional information.
Final alignment	Docking IR + IMU	Reduce positional and angular error before mechanical docking.

The UWB–IMU and IR systems overlap rather than hand over cleanly: UWB and IMU produce the relative-state estimate needed to manoeuvre into the docking region, after which IR supplies the high-resolution local feedback that final mechanical alignment requires. The transition threshold between stages, the alignment-error formulation, the sensor arrangement, the control strategy and the mechanical docking conditions are all to be fixed experimentally.

7.5 Docking interface

Magnetic circuit and holding-force model. An electropermanent magnet pairs a hard (AlNiCo-type) and a semi-hard (NdFeB-type) magnet sharing a common flux path. A short current pulse through a coil around the semi-hard element reverses its polarity. With the two elements aligned their fields add and flux crosses the air gap to a mating face; opposed, the fields cancel inside the circuit and negligible flux reaches the gap. Because the magnetic state is set by remanence rather than by sustained current, holding force does not scale with continuous power draw. That is the property Tosun et al. exploit in the EP-Face [**?**], and the reason the standby-power argument of Section **??** holds.

For a single pole pair with air-gap flux density B_g and pole face area A, the attractive force follows from the Maxwell stress tensor evaluated at the gap,

$$F \approx \frac{B_g^2 A}{2\mu_0}, \qquad \mu_0 = 4\pi \times 10^{-7}\ \text{H/m}. \tag{21}$$

The EP-Face ring places n such pole pairs around the connector rather than one central magnet (Figure **??**). Treating the poles as acting in parallel,

$$F_{\text{total}} \approx \frac{n\, B_g^2 A_{\text{pole}}}{2\mu_0}. \tag{22}$$

The reported $\approx 90\,\text{N}$ figure for SMORES-EP [**?**, **?**] is therefore consistent with several small, low-flux poles rather than one large one — which is also what keeps the switching pulse energy low, since stored magnetic energy in each gap scales with pole volume rather than directly with total holding force. Equation (**??**) is the quantity our own connector sizing must reproduce or exceed once the face dimensions are fixed by the module envelope.

Cantilever moment capacity. Tilt is the strongest degree of freedom because it combines both motors' torque, which is what lets it carry the cantilever load in a chain. The connector has to survive that load too: it is what keeps the cantilevered chain attached to its supporting module in the first place.

Model a horizontal chain of k cantilevered modules extending from a fixed dock as point loads mg at module centres, at distances $d_1, \ldots, d_k$ from the supporting face. The bending moment the connector must react is

$$M = \sum_{i=1}^{k} m\, g\, d_i. \tag{23}$$

A distributed ring of magnets resists an applied moment the way a bolt-circle flange does, pulling one side of the ring into greater compression and the opposite side toward separation, with the reaction distribution approximately linear across the diameter. For a ring of effective radius r the moment capacity before any pole separates is

$$M_{\max} \approx F_{\text{total}}\, r, \tag{24}$$

which is the uniform-ring reduction of an integral of local pole force times moment arm. This is why a distributed genderless connector was preferred over a single central magnet: a central magnet's moment capacity is set only by friction at the contact face, not by a ring radius, and is considerably weaker for the same axial holding force.

What (**??**) and (**??**) leave open is the design constant that the torque analysis behind the obstacle table of Section **??** must close: the maximum chain length k, and the maximum obstacle span the arch or cantilever configuration can carry, before M exceeds $M_{\max}$ at whichever fails first, the weakest joint or the connector. That comparison, not an assumed value, is what should set the held-out parameter boundary.

Alignment tolerance budget. The EP-Face acceptance window — roughly 4 mm of separation and 7 mm of lateral misalignment [**?**] — is a margin, not a target. The positioning and alignment system must deliver docking-attempt error inside that window with high confidence, not merely on average. Treating the UWB/IMU macro-positioning error, the IR micro-alignment error and mechanical backlash as approximately independent, they combine at contact in root-sum-square:

$$\sigma_{\text{total}} = \sqrt{\sigma_{\text{UWB/IMU}}^2 + \sigma_{\text{IR}}^2 + \sigma_{\text{backlash}}^2}. \tag{25}$$

A conventional 3σ margin then requires $3\sigma_{\text{total}} \leq 7\,\text{mm}$ laterally and $3\sigma_{\text{total}} \leq 4\,\text{mm}$ in separation at the moment the connector is commanded to engage.

That gives a concrete acceptance test for the cascaded positioning system during Phase B bring-up: log the achieved σ at each stage transition — UWB hand-off to IR, IR hand-off to contact — and confirm the budget closes before relying on the $\geq 90\,\%$ dock/undock criterion of Section **??**. A docking system that only occasionally falls inside the acceptance window cannot clear that bar regardless of how strong the connector itself is.

7.6 Task-oriented planning

Formal problem statement. Configuration selection takes its input in one of two forms: a structured task-requirement vector, or raw sensor data from which that vector must first be estimated. Formalising the first case gives the baseline the rule-based library has to satisfy.

Let the task requirement be a vector $t \in \mathbb{R}^p$; for our obstacle classes $p = 1$, being gap width, step height or passage width. Let $\mathcal{L} = \{c_1, \ldots, c_m\}$ be the finite library of pre-verified configurations — single module, linear chain, arch/cantilever, and the optional non-planar fourth. Each configuration carries a capability function $\mathrm{Cap}(c_i) \in \mathbb{R}^p$ giving the largest obstacle instance it can clear, such as an arch's maximum gap width derived from (**??**). The rule-based selector is then

$$c^* = \operatorname*{arg\,min}_{c_i \in \mathcal{L} \,:\, \mathrm{Cap}(c_i) \geq t} \mathrm{Cost}(c_i), \tag{26}$$

subject to the feasibility constraint that c_i's connection graph is a tree, since the sequencer assembles trees only and cannot close a cycle; with the helper-module manoeuvre implemented, lateral (L/R) edges are routable and are no longer excluded, at the cost of reserving a module per lateral action. Taking Cost as module count — or equivalently assembly depth, since parallel assembly time scales with tree depth rather than module count — reduces selection to a bounded linear scan of $O(|\mathcal{L}|\, p)$, evaluated once per obstacle encounter. That is deliberately far cheaper than the graph-matching alternatives surveyed in Section **??**, which is consistent with treating those as later stages rather than as the baseline.

Why the capability function must be closed-form. The 80 % selection target is measured explicitly *including held-out obstacle parameters*: the selector must generalise past the training range, not memorise it. A library whose $\mathrm{Cap}(c_i)$ values are fitted post hoc from simulation trials at a few sampled parameter values cannot be trusted to extrapolate. A library whose values are derived from the underlying mechanics — (**??**) for the arch, an analogous static-equilibrium relation for the step — extrapolates by construction, because the same equation applies outside the sampled range.

This is the reason for developing the moment model before the simulation campaign rather than after. The model *is* the capability function the selector consults, and its validity outside the training range is precisely what the held-out success criterion tests.

Graph representation and the GNN extension. A graph neural network is proposed as the second stage, once the task space outgrows a fixed library, operating directly on the Face2Con connection-graph notation already adopted for the reconfiguration planner [**?**]. Framing it now rather than deferring it entirely clarifies what the rule-based stage must log to make the transition possible.

Represent a configuration as a graph $G = (V, E)$, one node per module and one edge per active docking connection, each edge labelled with the relative face and orientation. A message-passing network updates each node embedding by aggregating over its neighbours,

$$h_v^{(k+1)} = \phi\left(h_v^{(k)}, \bigoplus_{u \in \mathcal{N}(v)} \psi\Big(h_v^{(k)}, h_u^{(k)}, e_{uv} \Big) \right), \tag{27}$$

for learned ϕ, ψ and a permutation-invariant aggregator $\bigoplus$ such as sum or max. Permutation invariance is the operative property. It means the same network evaluates a configuration correctly regardless of which physical module ID occupies which graph position, so the network does not need retraining when the Kuhn–Munkres task assignment changes which module plays which role.

A configuration's final embedding, pooled over all $h_v^{(K)}$, is trained to regress $\mathrm{Cap}(c)$ or to classify suitability directly against t, replacing the library lookup once the library grows too coarse or too large to enumerate. Because this shares the input representation of the existing reconfiguration pipeline, introducing the second stage needs no new perception-to-planner interface — only the selection function changes, which is the property the phased design is meant to preserve.

7.7 Built-in camera for perception

What the perception stack has to deliver. To decide whether a region is traversable, the robot needs local terrain information of two different kinds: where the surfaces are, and what they are made of. The system therefore carries two complementary sensors — a multi-zone time-of-flight (ToF) sensor for geometry and a compact monocular RGB camera for appearance. In the prototype both are mounted on the leader module only, and raw data is carried over a wired link to an external computer. Decoupling the sensing hardware from the computing hardware in this way lets the perception algorithms be developed and evaluated in Phase C without a high-performance compute unit on every module, which is the Phase D problem.

Geometric perception from a multi-zone ToF sensor. A ToF sensor recovers distance from the round-trip time of an emitted light pulse,

$$d = \frac{c\,\Delta t}{2}, \tag{28}$$

where c is the speed of light and Δt the round-trip time. A multi-zone device resolves this separately over discrete regions of its field of view rather than returning one number, so an 8×8 part yields a compact depth matrix

$$\mathbf{D} = \begin{bmatrix} d_{1,1} & \cdots & d_{1,8} \\ \vdots & \ddots & \vdots \\ d_{8,1} & \cdots & d_{8,8} \end{bmatrix}. \tag{29}$$

This is the useful middle ground: enough structure to measure obstacle proximity, depth discontinuity and local surface variation — the quantities the governing obstacle parameters of Section **??** are read from — without the cost of building and carrying a dense point cloud. A part such as the VL53L5CX provides 64-zone ranging in a package small enough for a module of this size, and 64-zone ToF sensing has already been shown to support obstacle avoidance on severely resource-constrained platforms [**?**].

Visual perception from a monocular camera. The camera is not there to run general-purpose object detection. Its role is to supply the appearance information the ToF sensor cannot: colour, texture and surface type. A lightweight model extracts broad visual features or coarse

surface categories relevant to traversability — distinguishing smooth flooring from loose gravel, for instance, while the ToF sensor supplies the corresponding slope and depth. Which features and which classifier are used is left to be fixed during development, against the compute budget and the training data actually available.

Fusing the two. The two modalities are combined into a single local terrain representation,

$$F_{\text{terrain}} = f(G_{\text{ToF}},\, V_{\text{RGB}})\,, \tag{30}$$

with G_{ToF} the geometric features from the depth matrix and V_{RGB} the visual features from the image. Fusing geometry with semantics is established practice in terrain analysis for legged navigation [**?**]; what differs here is the sensing budget. That work relies on a substantial sensing payload, whereas this system has to reach a usable traversability estimate from a low size, weight and power configuration that a single module can carry.

Component	**Sensor**	**Main function**	**Processing**
Geometric perception	8×8 multi-zone ToF	Depth matrix; obstacle proximity, discontinuities, surface variation	External computer
Visual perception	Monocular RGB camera	Surface appearance, texture, semantic visual features	External computer
Perception fusion	ToF + RGB	Fused local terrain and traversability representation	External computer
Communication	Wired link	High-bandwidth transfer of raw sensor data	Wired interface

This is the Phase C starting architecture, and it is deliberately the conservative one: it keeps the sensing hardware inside the module envelope while leaving computation off-board, so that the perception work is not blocked on the compute-module selection. Sensor placement, calibration, data synchronisation, the feature-extraction pipeline, the fusion function f and the decision logic above it are developed and evaluated as part of the methodology.

7.8 Compute module

Camera geometry. Perception rests on the pinhole model, in which a point $\mathbf{X}$ in the camera frame projects to image coordinates through the intrinsic matrix

$$\mathbf{x} = \mathbf{K}\,\mathbf{X}, \qquad \mathbf{K} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}, \tag{31}$$

with calibration required to recover $\mathbf{K}$ and the lens distortion coefficients before any metric measurement is trusted.

Pose from one image. The relative pose of a docking face carrying a planar fiducial is a perspective-n-point problem: four known coplanar points and their image projections determine

the full six-degree-of-freedom pose of the target from a single frame, with no stereo baseline and no depth sensor. This is what allows a single monocular camera to serve alignment.

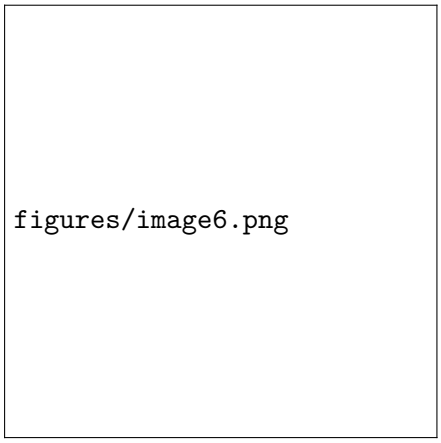
The real-time constraint is detection rate against control rate. A comparable modular-robot system was limited to roughly 4 Hz by the detection step [**?**], which is slow enough to dominate the docking latency budget derived above. The theory required therefore includes the computational cost of the detection algorithm and how it scales with image resolution and with the size of the search region — restricting the search to a predicted neighbourhood is the standard route to raising the rate without new hardware.

Configuration selection as classification. Selecting a configuration is treated as a mapping from the perceived obstacle state to an entry in the design library, which makes it a classification problem with a small, fixed label set — the reason a rule-based baseline is defensible before any learned model is introduced.

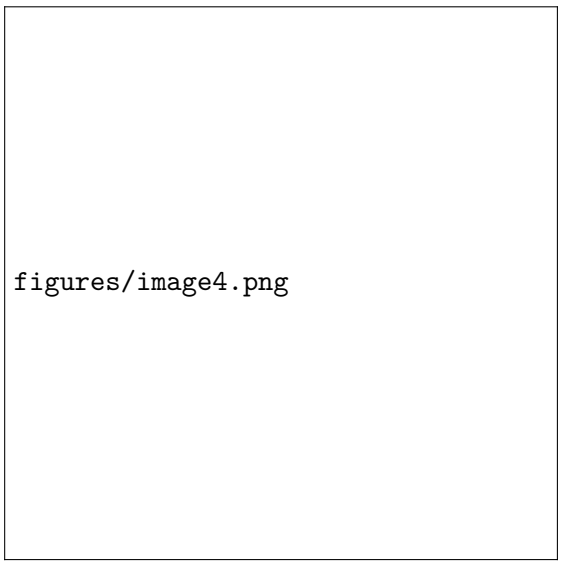
Inference on an embedded target. Running that model on the module rather than off-board requires quantisation from floating point to fixed-point or integer arithmetic to fit the memory and the arithmetic units of an embedded NPU. The relevant theory is the accuracy cost of quantisation against the latency and power saved, since this is the trade that decides whether Phase D is achievable on the candidate compute modules.

7.9 Reconfiguration planning

Configuration representation. A configuration is a graph $G = (V, E)$, the vertices being modules and each edge carrying Face, Face2Con and Orientation, with Face $\in$ {LEFT (L), RIGHT (R), TOP (T), BOTTOM (B)}. A connection is therefore described by the pair of connectors it joins and their relative orientation, not by a single label per module.



(a) Module face labelling.



(b) Connection graph of a configuration.

Figure 5: Connection-graph notation for a module configuration, adapted from Liu and Yim [**?**]. T = top, R = right, L = left, B = bottom.

Target configurations are restricted to tree topologies — no closed loops — an assumption inherited from Liu et al. [**?**] and acceptable because all four obstacle morphologies are trees.

Interface to configuration selection. The planner receives a target configuration, not an obstacle. Upstream, the established mechanism is a property-indexed library [**?**, **?**]: each entry pairs a configuration C and a behaviour B_C with behaviour properties P_b (action, speed, height, width) and environment properties P_e (stair height, tunnel height, gap width, ground roughness); perception classifies the environment and the planner returns entries satisfying every required property. Daudelin et al. [**?**] instantiate this with four environment classes — free, tunnel, stairs, high — derived from a 3D occupancy grid. Two consequences follow for this subsystem. First, the reachability filter below must be applied at query time, so that configurations containing cycles are never returned. Second, the library query in the source work is blind to reconfiguration cost, while the assignment stage below computes exactly that; exposing that cost as a further selection criterion is an available extension where two library entries would otherwise both satisfy the query.

Reachability filter. With lateral docking implemented, the tree-only assumption is the sole remaining restriction on the published configuration set.

Configuration	**Topology**	**Reachable**	**Assembly requires**
Snake, StairClimber	Linear chain	Yes	TOP–BOTTOM actions only
Proboscis	Chain on a base	Yes	Helper module for the base's lateral edges
Car, DoubleDriver, SwerveLifter	Modules abreast	Yes	Helper module
Scorpion	Branching tree	Yes	Helper module
RollingLoop	Closed cycle	No	Excluded by the tree-only assumption

Only cycles are unreachable. The practical constraint is therefore not topology but module budget: every lateral edge consumes a helper module for the duration of its action, so a configuration with several lateral edges at the same depth serialises where a chain would not.

Task assignment. The target tree is unfolded onto the plane and its graph centre selected as the target root g_τ by a linear-time root-module search; every target module is expressed as a relative pose $[x, y, \theta]$ with respect to g_τ. On the physical side the module nearest the group centroid,

$$m_\tau \;=\; \arg\min_{m_i \in M} \|o_i - o_c\|\,, \tag{32}$$

is taken as the physical root, and every module expressed relative to it. The cost matrix is $C_{ij} = \|\tilde{o}_i - \bar{o}_j\|$, and Kuhn–Munkres returns the bijection $f : V \to M$ minimising total travel. Complexity is $O(n^3)$ — sub-millisecond at $n \leq 6$, confirming that the algorithm earns its place by minimising physical motion rather than computation.

Proposed extension. The inherited cost is orientation-blind. We will evaluate

$$C_{ij} \;=\; \|\tilde{o}_i - \bar{o}_j\| \;+\; \lambda\,|\Delta\theta|\,, \tag{33}$$

penalising net reorientation, since in-place turning on a differential-drive base is slower and less repeatable than translation; λ is calibrated from measured turn-versus-translate timing.

Assembly sequencing. At depth d, each target module v maps to $m = f(v)$ and yields an action $(m, c, \tilde{m}, \tilde{c}')$: connect m's connector c to parent $\tilde{m}$'s connector $\tilde{c}'$. Same-depth actions are mutually independent and form one action group. One exception applies at the root: because it is unfixed and can be displaced, actions on the root's LEFT/RIGHT faces precede those on its TOP/BOTTOM faces.

Navigation [?, ?]. The environment is a graph $G(V, E)$ whose vertices are path segments carrying a length (shortest-route search) and a width (passability); in our case the 10 cm grid of Liu et al. supplies this graph directly and width reduces to a binary footprint test, so we inherit the algorithm without its map-building stage. Each module is a differential-drive vehicle,

$$\dot{p} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \cos\theta & 0 \\ \sin\theta & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix}, \tag{34}$$

with v and ω from the LEFT and RIGHT wheel DoFs. Modules are planned one at a time in priority order by the Single Robot Route Planner, an A* search over G that updates a vertex potential only if the vertex is unoccupied at that time step, querying a Route Coordinator holding the already-planned routes. Routes are stored as timed vertex locks, which permits explicit waiting, permits a module to hold several vertices at a crossing, and — because locks overlap slightly in time — makes a position swap register as a collision rather than passing undetected.

On reaching an occupied vertex the Multi Robot Collision Resolver copies the affected vertices into a temporary subgraph, leaving the original potentials intact, and tries in order: backtracking (retreat to the first vertex not shared with the blocker and wait), avoid-at-crossing (pull into a side branch), avoid-at-start and avoid-at-goal (for modules that cannot back up or are blocking from their goal), and push (displace the waiting vertex recursively when no branch is free). If routing still fails, speed rescheduling lowers the maximum speed of the module colliding most often with the failing one, and then priority rescheduling swaps the priorities of the two modules with the most unresolved collisions — selecting by collision count rather than at random avoids swaps between modules not actually in conflict. The Route Generator then emits the routing table with its per-segment preconditions, and each module follows its route under a PID path-following controller, publishing its segment index so peers can evaluate preconditions; segments overlapping at a crossing are locked together. A reference implementation exists as the ROS package `tuw_multi_robot_router`, which we will evaluate against reimplementing on our stack.

Pose adjustment. With pose error (x_i', y_i', θ_i') in the parent's frame, TOP/BOTTOM connections drive $y_i', \theta_i' \to 0$ and relax x_i'; LEFT/RIGHT connections drive $x_i', \theta_i' \to 0$ and relax y_i'. For the TOP/BOTTOM case,

$$\begin{bmatrix} v \\ \omega \end{bmatrix} = \begin{bmatrix} \sin\theta_i' & 0 \\ 0 & 1 \end{bmatrix}^{-1} K \begin{bmatrix} -y_i' \\ -\theta_i' \end{bmatrix}, \qquad K = \mathrm{diag}(2, 1), \tag{35}$$

relaxing one dimension to obtain full actuation over the remaining two under a nonholonomic base. The relaxed coordinate is unregulated and may diverge, risking collision on approach; we bound this by gating the approach on $|x_i'|$ staying inside a measured envelope and by capping pose-adjustment duration.

Approach. For a TOP/BOTTOM connection the module drives straight at the target connector until the EP connectors engage, pushing the parent into contact where necessary.

A LEFT/RIGHT connection cannot be closed this way, because a differential-drive base cannot translate along its wheel axis into the mating pose. The action is therefore executed with a *helper module*: the helper docks to the child's TOP face, lifts the child clear of the ground, carries it into alignment with the parent's lateral connector, sets it down so the EP faces engage, then releases its own TOP connection and withdraws. The helper is unavailable for other actions throughout, so the sequencer treats a lateral action as consuming two modules rather than one when composing an action group.

Module localisation. Poses are required in a common frame by task assignment, by the route planner and by the PID path follower. Phase A uses the arrangement of Daudelin et al. [**?**]: an RGB-D camera running RTAB-MAP SLAM builds a 3D occupancy grid of the environment, while AprilTag fiducials on each module are tracked by a downward-facing camera covering the reconfiguration area, giving module pose during navigation and the final approach. Docking tolerance is therefore set by fiducial tracking accuracy rather than by odometry drift, which is what makes the capture-envelope threshold in pose adjustment measurable rather than assumed.

Assumptions and validation.

Assumption	**Mitigation**
Module poses available in a common frame — required by task assignment, route planning and path following alike	AprilTag fiducials tracked by a downward-facing camera over the reconfiguration area, with RGB-D SLAM for the environment map, following Daudelin et al. [**?**]; external perception, consistent with Phase A scope. Transition to onboard localisation scoped to Phase D.
A helper module is free whenever a lateral action is scheduled	Sequencer reserves a module per lateral action when composing an action group; groups exceeding the module budget are split across depths.

Assumption	Mitigation
Routing succeeds for every action group	Speed then priority rescheduling; on continued failure, sequential execution of the group.
Contact alignment within the EP capture envelope	Characterise experimentally; use as the convergence threshold for pose adjustment.
Flat, obstacle-free staging area	Restrict reconfiguration to a cleared area, as in the $0.75 \times 0.5\,\mathrm{m}$ tracked reconfiguration zone of Daudelin et al. [**?**]; on-obstacle reconfiguration is out of scope.
Each docking action succeeds	Verify via EP connector feedback and topology self-check; retract and retry, then re-plan the remainder from the current partial topology.

Validation proceeds in four stages: assignment and sequencing against hand-computed optima in simulation for $n = 3$–6; routing against contrived conflicts (head-on, crossing, position swap, goal-blocking) to confirm that each resolver strategy triggers; single-pair docking success rate and residual alignment error on hardware over at least 20 trials; and then the full pipeline in simulation and on hardware.

7.10 Simulation and assembled locomotion

Docking in simulation. A connection is a weld equality constraint between two module bodies, activated when the connector frames fall inside the capture envelope and deactivated on command. Joint compliance — representing the electropermanent connector's finite stiffness — is obtained through the constraint's solver reference and impedance parameters rather than by any change of constraint type, so a docked pair is compliant without ceasing to be a single kinematic structure.

Randomisation ranges. The initial values below are placeholders, to be replaced by the residual alignment error measured in the docking trials above.

Parameter	Initial range
Per-interface translation offset	$\pm 5\,\mathrm{mm}$
Per-interface orientation offset	$\pm 3°$
Joint angles at handover	*[TBD — set from the docking controller's terminal state]*
Approach angle to obstacle	*[TBD]*
Actuator backlash	*[TBD — measure on the prototype]*
Connector compliance	*[TBD — measure on the prototype]*

Configuration models. Each assembled configuration is exported as its own model, generated from its connection graph: modules become links, and each connection becomes a compliant joint at the mating faces with stiffness and damping set from the measured connector characteristics. The generator consumes the same (FACE, FACE2CON, ORIENTATION) edge description the planner uses, so a change to the module model propagates to every configuration without hand-editing. Configurations are trained as complete robots on these models; the separate weld-constraint scene is retained only for simulating the reconfiguration process itself.

Policy training. PPO [**?**], one policy per contact-rich configuration, each treated as an independent robot with no parameter sharing between configurations. Observation: joint states, IMU, and obstacle-relative pose. Action: joint position targets. Reward: progress along the traversal direction, less energy expenditure, with termination on tip-over or timeout. Training runs in MJX; the trained policy is exported and wrapped as a ROS 2 node for deployment.

Baseline and evaluation protocol. Each configuration has a scripted, open-loop gait in the manner of PolyBot's table-driven gaits [**?**]. Both controllers are evaluated over the same trial count on the same obstacle, reporting traversal success rate and traversal time; a learned policy is adopted only where it beats the scripted gait on both, in simulation and then on hardware. The scripted gait remains the deployed fallback where it does not.

8 Expected Project Outcome

The outcome of this project is a small fleet of prototype modules — four to six, the count itself an output of the actuator sizing of Section **??** rather than an input — together with a physics-validated twin and the measured data supporting both. What the hardware is meant to show is not a deployable system but a demonstrated level of task-oriented behaviour: presented with an obstacle it was not given in advance, the assembled robot estimates the parameter that matters, selects a configuration to suit it, reconfigures, and traverses. That loop is closed on one obstacle class in hardware; the claim that selection generalises beyond its calibration set is evaluated in simulation, at parameter values held out of the fit.

The scope is deliberately bounded. A prototype fleet of this size cannot establish reliability figures, and a terrestrial testbed cannot qualify anything for flight. What it can do is show that the decision layer works end to end on real hardware at small scale, and measure how far it degrades when the obstacle is one nobody specified in advance. Each deliverable below is stated with the standard it has to meet, so that success is decidable rather than a matter of opinion at the review.

8.1 Deliverables

#	Deliverable	Acceptance standard
1	Mechanically homogeneous MSRR modules, fully populated	Each module drives under its own control and docks to any other on any face.
2	EP-Face connector, characterised	Measured holding force, switching energy and area of acceptance, reported against the published SMORES-EP figures [**?**].
3	Physics-validated digital twin	Simulated torque, mass, friction and dock success reproduce the measured hardware values within the tolerances tabulated in Section 8.2.
4	Onboard perception pipeline	Obstacle class and governing parameter estimated from the module's own camera and ToF sensor, with no prior map.
5	Configuration-selection policy	Maps the estimated obstacle parameter to a target configuration. Judged on whether the **selected configuration traverses**, not on agreement with a labelled answer, and evaluated at parameter values held out of the capability fit — interpolated and extrapolated values reported separately.
6	Reconfiguration and traversal stack	Full perceive, select, reconfigure, traverse loop closed end-to-end on hardware.
7	Test arena and measurement protocol	Gap, step and narrow-passage obstacles at documented dimensions, with a repeatable trial procedure.
8	Documentation	Source, CAD, PCB files and reports sufficient for another group to rebuild a module.

8.2 Expected performance

The targets below are set against published results for comparable systems rather than chosen for roundness. The basis for each is given here, and Section 3.4 refers to this section rather than restating it.

Quantity	**Target**	**Basis**
Configuration-selection success, held-out parameters, in simulation	$\geq 80\,\%$	Between the 95 % and 73 % rates reported for four- and six-module clusters by the closest published learned reconfiguration policy [**?**]; our module count falls in that range.
Dock and undock from a pre-aligned start	18 of 20	Matches the 9 of 10 reported in [**?**] under fixed initial conditions, over twice the trials.
Dock and undock through the full navigate–adjust–approach sequence	14 of 20	Set below the pre-aligned figure because navigation and localisation add failure modes the connector cannot control; the single failure in [**?**] came from visual pose estimation, not from the connector.
Full autonomous loop, one obstacle class, hardware	7 of 10	Set below the docking rate because the loop compounds perception, selection and traversal.
Twin-to-hardware agreement on dock success	Within 10 points	The randomisation ranges of Section **??** are sized to absorb a mismatch of this order; a larger gap invalidates policies trained on the twin. To be revised once the first measurement exists.
Traversal success, learned policy against scripted gait	Improvement on success **and** time	The scripted gait is the baseline; a policy that does not beat it on both is not adopted.

On the selection target. The 80 % figure is measured end-to-end from the *estimated* obstacle parameter, not from a ground-truth value. This matters, because the decision itself is small — three obstacle classes with one governing parameter each, resolving to three or four configurations, and therefore a handful of thresholds that the torque analysis of Section **??** largely determines in advance. Evaluated on exact parameters the decision rule would be expected to score far above 80 %. The residual is dominated by estimation error from a monocular camera and an 8×8 time-of-flight grid with no prior map, which is where the difficulty in this subsystem actually lies.

Success is counted as **the selected configuration traversing the obstacle**, not as agreement with a labelled answer. Where two configurations both succeed at a given parameter value,

either choice is correct; scoring against a label would count a working choice as a failure, and would penalise the system hardest exactly where the ambiguity is genuine.

On the comparison to [?]. That result is for modular self-reconfigurable satellites performing path planning. The bridge to our target is module count and learning approach, not task domain — no closer learned-selection result appears to exist for terrestrial modular hardware. The transfer is stated here rather than left for the reader to notice.

On trial counts. Twenty trials for docking is a deliberate doubling of the ten used in [?]. Twenty trials distinguish a working connector from a marginal one; they do not resolve a few points of difference in success rate, and no claim of that precision is made. Targets are stated as counts rather than percentages so that the sample size stays visible.

On residual failure. Setting any bar at 100 % would not be honest. Daudelin et al. categorised 24 recorded failures of a SMORES-EP demonstration and attributed 10 of them (41.7 %) to hardware faults and 3 (12.5 %) to navigation [?] — causes no algorithm removes. A residual failure rate is a property of this class of system rather than a shortfall in this instance of it.

8.3 Advance over the baseline system

Daudelin et al. [**?**] is the reference point for this project's outcome: the closest published system to the loop we intend to close, on the same hardware lineage. The advance claimed is not in the reconfiguration machinery, which we inherit, but in **how the target configuration is arrived at**.

Stage	**Daudelin et al. [?]**	**This project**	**Why the difference matters**
Obstacle detection	Colour blob detection locates the object of interest	Monocular camera with an 8×8 multi-zone ToF sensor; geometric, no colour prior	A colour prior needs a known target and tuned lighting, neither available on a planetary surface or in rubble.
Environment representation	Classification into four discrete classes — free, tunnel, stairs, high — from a 3D occupancy grid	Continuous estimation of a governing parameter per obstacle class: gap width, step height, passage width	Binning discards magnitude, and magnitude decides which configuration works. A 10 cm step and a 30 cm step are one class but not one problem.
Configuration selection	Property lookup in a hand-authored library of configurations and behaviours [**?**]	Capability model over the governing parameter, calibrated in simulation against an analytically derived boundary, evaluated at held-out values	A library returns nothing for an environment no contributor anticipated. The authors of the baseline name "oversimplified environment classification" among their own stated limitations.
Sensing and computation	Carried sensor module with RGB-D, off-board high-level planning, module pose from a motion-capture system	Onboard camera, ToF and compute; ultra-wideband and inertial fusion for relative localisation, infrared and inertial feedback for short-range alignment	Removing the instrumented arena is what separates a laboratory demonstration from a deployable capability.

What is claimed, and what is not. The comparison is of mechanism, not of demonstrated scope. Daudelin et al. closed their loop across three hardware demonstrations in an environment not known in advance; this project targets one obstacle class in hardware at the Should tier of

Section 3.4, with the generalisation claim evaluated in simulation. We do not claim to exceed the baseline in demonstrated breadth within the span of Capstone Design Project I and II. The claim is narrower and testable: a selection layer built on continuous parameter estimation admits obstacle instances outside its calibration set, where a four-class classification followed by library lookup returns no answer at all. The held-out evaluation of Section 3.4 measures exactly that, which is why the held-out condition is written into the criterion rather than left as an aspiration.

Residual dependence on the baseline. Three components are adopted rather than improved upon, and the proposal does not obscure it: the Kuhn–Munkres task assignment and parallel self-assembly pipeline [**?**]; the connector and docking manoeuvre, including the helper-module lift for lateral connections [**?**]; and the property-indexed structure of the configuration library itself [**?**]. The contribution is the layer that decides what to ask that machinery for.

8.4 Split between Capstone Design Project I and II

Project I ends with a working modular robot that reconfigures on command: Phases A and B, external perception and computation, deliverables 1, 2, 3 and 7 delivered, and the docking and reconfiguration figures measured. The task-oriented layer exists but is driven from a stated task requirement rather than from the robot's own sensing.

Project II moves perception and then computation onto the module: Phases C and D, delivering deliverables 4, 5 and 6. The central claim of the project — that the robot decides for itself, with nothing off-board — is only demonstrated at the end of Project II. Project I is what makes that demonstration possible, and is scoped so that it stands on its own if the second stage is curtailed.

9 Resources Required

The project is funded to 200,000 THB, which covers module hardware, the connector, sensing and compute for the whole build. What follows is what the work needs rather than a costing; suppliers and prices are fixed during Phase A detailed design.

Hardware. Geared DC motors, microcontrollers and dual H-bridge drivers, magnetic encoders and IMUs for each module. Custom PCBs. Electropermanent magnet stock — AlNiCo and NdFeB with machined pole pieces. Sensing for the later phases: UWB modules, IR emitters and detectors, a monocular camera and a multi-zone ToF sensor. Compute modules for the coordinator only. Batteries and charging.

Facilities. FDM printing for module structure and machining. Electromagnetic field equipment and tooling for EP-Face characterisation — magnetising and demagnetising the AlNiCo core, holding force and switching energy. Electronics assembly and bench instrumentation. A tensile tester or calibrated load cell for connector characterisation.

Software. All open source or free for academic use: ROS 2 [**?**], Gazebo [**?**], MuJoCo and MJX [**?**], and PPO [**?**]. Isaac Sim is retained as an alternative if MuJoCo's sensor models prove inadequate. CAD and PCB tools are used in the design phase.

Data. There is no external dataset. Every figure the project relies on is measured from the real hardware. Training data is generated in simulation with randomisation ranges set from those measurements.

10 Summary and Benefit to Real Industry

This project builds a modular self-reconfigurable robot that chooses its own shape. Existing systems are told which configuration to form and then work out how to form it; ours perceives an obstacle, infers what capability is needed to pass it, selects a configuration, reconfigures, and traverses — using only the sensing and computation it carries. The work is staged over two semesters, from a single module driving under external supervision to a cluster that closes the whole loop on board, and is evaluated against three obstacle classes with targets drawn from published results for comparable systems.

The contribution is the selection layer and the evidence that it survives contact with hardware. Learning-based reconfiguration is almost entirely a simulation literature, and contact dynamics and magnetic docking are exactly what fails to transfer out of simulation. Building the twin against measured values, and then reporting where the twin and the hardware disagree, is as much a result as the traversal statistics.

10.1 Benefit to real industry

Space and planetary operations. Agencies are already funding modular reconfigurable hardware for on-orbit servicing and lunar surface work [**?**, **?**, **?**]. The constraint those programmes run into is the same one here: a robot several light-minutes away cannot be told which shape to adopt. Any progress on autonomous configuration selection transfers directly, and the terrestrial analogue arena is the standard way such systems are qualified before flight.

Search, rescue and disaster response. Collapsed structures present openings that no fixed-shape robot can pass, and the environment cannot be surveyed beforehand. A cluster that can split, pass a gap one module at a time, and reassemble addresses a capability gap that current USAR robots do not close.

Inspection of confined infrastructure. Pipes, ducts, tanks and cable runs share a shape: a small opening leading to a large interior. Inspection robots today are built per-geometry. A modular system that reconfigures to the geometry it finds replaces a family of single-purpose tools with one inventory of identical units — which also changes the spare-parts economics, since every module is a spare for every other.

Manufacturing. Reconfigurable fixturing and small-batch automation face the same argument in miniature: retooling is expensive because tooling is specific. This is the least developed of the four directions and we claim it only as an adjacency, not as a target application.

More immediately, the project produces something the department keeps: a documented, reproducible modular robot platform at a cost that a student team can absorb. Most MSRR hardware is either a national-agency investment or a fragile laboratory one-off, and the gap between those is where teaching and follow-on research actually happen.

11 Roles and Responsibilities

Responsibilities follow the ten workstreams of the Gantt chart in Section **??**. Each member owns a set of workstreams end to end: the design decisions within them, the corresponding sections of the report, and the verification evidence. Ownership is not sole execution — fabrication, assembly and trial weeks are shared across the team — but it fixes accountability for each subsystem, and it is the unit against which progress is tracked at each of the reporting milestones.

Member	**Workstreams**	**Responsibility**
Nat	W1, W2, W3, W10	**Module hardware and electronics.** Module geometry, DoF arrangement and drivetrain including the crown-gear differential; actuator sizing against the cantilever analysis. Inter-module radio link, message set and topology reporting. Custom PCB layout, motor drivers, joint-angle sensing, IMU fusion and firmware. In Phase D, the compute module and quantised onboard inference.
Jai	W4, W5	**Planning and simulation.** Reconfiguration planning — root selection, Kuhn–Munkres task assignment, parallel assembly sequencing and the prioritised route planner. The physics twin, its validation against measured hardware parameters, scripted gait baselines, and PPO training of assembled-locomotion policies with the randomisation ranges tied to measured docking error.
Prae	W6, W9	**Positioning and perception.** Cascaded relative positioning — UWB with IMU for the approach, infrared with IMU for final alignment — and the filter design behind it. From Phase C, the onboard perception stack: monocular camera and multi-zone ToF, terrain and traversability estimation feeding obstacle classification.
Aew	W7, W8	**Docking and task-oriented selection.** EP-Face connector design, build and characterisation — holding force, switching energy, area of acceptance — and the dock/undock trial campaign. Above it, the configuration-selection layer: the capability library, the rule-based baseline, and its evaluation against held-out obstacle parameters.

References

[1] Alciatore, D. G., & Histand, M. B. (2011). *Introduction to Mechatronics and Measurement Systems* (4th ed.). McGraw-Hill.

[2] An, X., Jia, Q., Chen, G., Liu, H., Liu, Y., & Wang, X. (2026). A new self-reconfigurable space modular robot SSMBOT: Design and performance analysis. *Advances in Space Research*, 77.

[3] Binder, B. (2017). *Spatio-temporal prioritized planning* [Diploma thesis, TU Wien]. https://doi.org/10.34726/hss.2017.40172

[4] Binder, B., Beck, F., König, F., & Bader, M. (2019). Multi robot route planning (MRRP): Extended spatial-temporal prioritized planning. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* (pp. 4133–4139).

[5] Buchi, B., Mabed, H., Lassabe, F., et al. (2025). Machine learning for modular robots self-reconfiguration problem. *Journal of Ambient Intelligence and Humanized Computing*, 16, 347–359. https://doi.org/10.1007/s12652-024-04949-x

[6] Castaño, A., Behar, A., & Will, P. M. (2002). The Conro modules for reconfigurable robots. *IEEE/ASME Transactions on Mechatronics*, 7(4), 403–409.

[7] Craig, J. J. (2005). *Introduction to Robotics: Mechanics and Control* (3rd ed.). Pearson Prentice Hall.

[8] Dai, Y., He, S., Nie, X., Rui, X., Li, S., & He, S. (2024). Research on reconfiguration strategies for self-reconfiguring modular robots: A review. *Journal of Intelligent & Robotic Systems*, 110(47). https://doi.org/10.1007/s10846-024-02067-6

[9] Daudelin, J., Jing, G., Tosun, T., Yim, M., Kress-Gazit, H., & Campbell, M. (2018). An integrated system for perception-driven autonomy with modular robots. *Science Robotics*, 3(23), eaat4983. https://doi.org/10.1126/scirobotics.aat4983

[10] Davey, J., Kwok, N., & Yim, M. (2012). Emulating self-reconfigurable robots: Design of the SMORES system. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* (pp. 4464–4469).

[11] Dobreva, N., Blazquez, E., Grover, J., Izzo, D., Qin, Y., & Dold, D. (2025). Decentralised self-organisation of pivoting cube ensembles using geometric deep learning. *arXiv:2509.03140*.

[12] Dutta, A., Dasgupta, P., & Nelson, C. (2019). Distributed configuration formation with modular robots using (sub)graph isomorphism-based approach. *Autonomous Robots*, 43(4), 837–857. https://doi.org/10.1007/s10514-018-9759-9

[13] Haddeler, G., Chuah, M. Y. M., You, Y., Chan, J., Adiwahono, A. H., Yau, W. Y., & Chew, C.-M. (2022). Traversability analysis with vision and terrain probing for safe legged robot navigation. *Frontiers in Robotics and AI*, 9, 887910. https://doi.org/10.3389/frobt.2022.887910

[14] Higgins, W. T. (1975). A comparison of complementary and Kalman filtering. *IEEE Transactions on Aerospace and Electronic Systems*, AES-11(3), 321–325.

[15] Jing, G., Tosun, T., Yim, M., & Kress-Gazit, H. (2016). An end-to-end system for accomplishing tasks with modular robots. In *Robotics: Science and Systems XII*. https://doi.org/10.15607/RSS.2016.XII.025

[16] Jing, G., Tosun, T., Yim, M., & Kress-Gazit, H. (2018). Accomplishing high-level tasks with modular robots. *Autonomous Robots*. https://doi.org/10.1007/s10514-018-9738-1

[17] Khoshnevis, B., Will, P., & Shen, W.-M. (2003). Highly compliant and self-tightening docking modules for precise and fast connection of self-reconfigurable robots. In *IEEE International Conference on Robotics and Automation (ICRA)*.

[18] Knaian, A. N. (2010). *Electropermanent magnetic connectors and actuators: Devices and their application in programmable matter* [Doctoral dissertation, Massachusetts Institute of Technology].

[19] Koenig, N., & Howard, A. (2004). Design and use paradigms for Gazebo, an open-source multi-robot simulator. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* (pp. 2149–2154).

[20] Kojcev, R., Etxezarreta, N., Hernández, A., & Mayoral Erle, V. (2018). Hierarchical learning for modular robots. *arXiv:1802.04132*.

[21] Lan, G., Benito-Picazo, J., Roijers, D. M., Dominguez, E., & Eiben, A. E. (2018). Real-time robot vision on low-performance computing hardware. In *15th International Conference on Control, Automation, Robotics and Vision (ICARCV)* (pp. 1959–1965). IEEE. https://doi.org/10.1109/ICARCV.2018.8581288

[22] Li, H., Wang, H., Cui, L., Li, J., Wei, Q., & Xia, J. (2022). Design and experiments of a compact self-assembling mobile modular robot with joint actuation and onboard visual-based perception. *Applied Sciences*, 12(6), 3050. https://doi.org/10.3390/app12063050

[23] Liang, G., Luo, H., Li, M., Qian, H., & Lam, T. L. (2020). FreeBOT: A freeform modular self-reconfigurable robot with arbitrary connection point — design and implementation. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* (pp. 6506–6513).

[24] Liang, G., Wu, D., Tu, Y., & Lam, T. L. (2025). Decoding modular reconfigurable robots: A survey on mechanisms and design. *International Journal of Robotics Research*. https://doi.org/10.1177/02783649241283847

[25] Liang, G., Ijspeert, A. J., Yim, M., & Lam, T. L. (2026). Modular reconfigurable robots: Towards on-demand multifunctional applications. *Science Robotics*. https://doi.org/10.1126/scirobotics.adz1999

[26] Liu, C., & Yim, M. (2017). Configuration recognition with distributed information for modular robots. In *International Symposium on Robotics Research (ISRR)*. Robotics Research, Springer Proceedings in Advanced Robotics (pp. 967–983). https://doi.org/10.1007/978-3-030-28619-4_65

[27] Liu, C., Whitzer, M., & Yim, M. (2019). A distributed reconfiguration planning algorithm for modular robots. *IEEE Robotics and Automation Letters*, 4(4), 4231–4238. https://doi.org/10.1109/LRA.2019.2930432

[28] Liu, C., Lin, Q., Kim, H., & Yim, M. (2023). SMORES-EP, a modular robot with parallel self-assembly. *Autonomous Robots*, 47(2), 211–228. https://doi.org/10.1007/s10514-022-10078-1 (Preprint *arXiv:2104.00800*, cited in some sections as Liu et al., 2021.)

[29] Liu, B., Ye, D., Yao, Z., & Sun, Z. (2025). A goal-oriented reinforcement learning-based path planning algorithm for modular self-reconfigurable satellites. *IFAC-PapersOnLine*. (Preprint *arXiv:2505.01966*.)

[30] Liu, H., Liu, Y., Yan, Y., Liang, Z., Liu, J., An, X., Nie, J., Xia, D., & Chen, G. (2026). Reconfiguration sequence planning algorithm of modular robot based on maximum similarity matching. *Measurement Science and Technology*. https://doi.org/10.1088/1361-6501/ae4f13

[31] Liu, J. G., Wang, Y. C., Li, B., et al. (2007). Center-configuration selection technique for the reconfigurable modular robot. *Science in China Series F: Information Sciences*, 50(5), 697–710.

[32] Liu, Y., Zheng, Q., Tu, Y., Gao, Y., Liang, G., & Lam, T. L. (2025). Configuration-adaptive visual relative localization for spherical modular self-reconfigurable robots. In *IEEE International Conference on Robotics and Automation (ICRA)*, Atlanta.

[33] Macenski, S., Foote, T., Gerkey, B., Lalancette, C., & Woodall, W. (2022). Robot Operating System 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66), eabm6074.

[34] Madgwick, S. O. H., Harrison, A. J. L., & Vaidyanathan, R. (2011). Estimation of IMU and MARG orientation using a gradient descent algorithm. In *IEEE International Conference on Rehabilitation Robotics (ICORR)*.

[35] Mahony, R., Hamel, T., & Pflimlin, J.-M. (2008). Nonlinear complementary filters on the special orthogonal group. *IEEE Transactions on Automatic Control*, 53(5), 1203–1218.

[36] Meng, Y., Zhang, Y., & Jin, Y. (2011). Autonomous self-reconfiguration of modular robots by evolving a hierarchical mechanochemical model. *IEEE Computational Intelligence Magazine*, 6(1), 43–54.

[37] Mishra, A., Santra, S., Neppel, E., Rossi Lombardi, E. M., Karimov, S., Uno, K., & Yoshida, K. (2025). Multi-modal decentralized reinforcement learning for modular reconfigurable lunar robots. *arXiv:2510.20347*.

[38] Müller, H., Niculescu, V., Polonelli, T., Magno, M., & Benini, L. (2023). Robust and efficient depth-based obstacle avoidance for autonomous miniaturized UAVs. *IEEE Transactions on Robotics*, 39(6), 4935–4951. `https://doi.org/10.1109/TRO.2023.3315710`

[39] Murata, S., Yoshida, E., Kamimura, A., Kurokawa, H., Tomita, K., & Kokaji, S. (2002). M-TRAN: Self-reconfigurable modular robotic system. *IEEE/ASME Transactions on Mechatronics*, 7(4), 431–441.

[40] Pathak, D., Lu, C., Darrell, T., Isola, P., & Efros, A. A. (2019). Learning to control self-assembling morphologies: A study of generalization via modularity. In *Advances in Neural Information Processing Systems (NeurIPS)* (Vol. 32, pp. 2292–2302).

[41] Peng, X. B., Andrychowicz, M., Zaremba, W., & Abbeel, P. (2018). Sim-to-real transfer of robotic control with dynamics randomization. In *IEEE International Conference on Robotics and Automation (ICRA)* (pp. 3803–3810).

[42] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv:1707.06347*.

[43] Siciliano, B., Sciavicco, L., Villani, L., & Oriolo, G. (2009). *Robotics: Modelling, Planning and Control*. Springer.

[44] Spröwitz, A., Pouya, S., Bonardi, S., van den Kieboom, J., Möckel, R., Billard, A., Dillenbourg, P., & Ijspeert, A. J. (2009). Roombots: Reconfigurable robots for adaptive furniture. *IEEE Computational Intelligence Magazine*.

[45] Stoy, K., Brandt, D., & Christensen, D. J. (2010). *Self-reconfigurable robots: An introduction*. MIT Press.

[46] Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., & Abbeel, P. (2017). Domain randomization for transferring deep neural networks from simulation to the real world. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* (pp. 23–30).

[47] Todorov, E., Erez, T., & Tassa, Y. (2012). MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* (pp. 5026–5033).

[48] Tosun, T., Davey, J., Liu, C., & Yim, M. (2016). Design and characterization of the EP-Face connector. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* (pp. 45–51).

[49] Tsai, L.-W. (1999). *Robot Analysis: The Mechanics of Serial and Parallel Manipulators*. John Wiley & Sons.

[50] Tu, Y., Liang, G., Wu, D., Li, X., & Lam, T. L. (2025). Locomotion and self-reconfiguration autonomy for spherical freeform modular robots. *International Journal of Robotics Research*. `https://doi.org/10.1177/02783649251360360`

[51] Matsuo, K., Makabe, T., Yanokura, I., Kojima, K., Okada, K., & Inaba, M. (2026). TYCOON: Modular robot capable of torque synthesized collaboration with complementary intermittent gear mechanism. *Advanced Robotics*, 40(4), 222–246. `https://doi.org/10.1080/01691864.2026.2618831`

[52] Wang, B., & Stokes, A. A. (2025). Self-aligning EPM connector: A versatile solution for adaptive and multi-modal interfaces. *arXiv:2508.16008*.

[53] Wei, H., Chen, Y., Tan, J., & Wang, T. (2011). Sambot: A self-assembly modular robot system. *IEEE/ASME Transactions on Mechatronics*, 16(4), 745–757. https://doi.org/10.1109/TMECH.2010.2085009 (Cited in some sections as Wei et al., 2010.)

[54] Wei, R., Liu, Y., Dong, H., Zhu, Y., & Zhao, J. (2023). A graph-based hybrid reconfiguration deformation planning for modular robots. *Sensors*, 23(18), 7892. https://doi.org/10.3390/s23187892

[55] Whitman, J., Travers, M., & Choset, H. (2023). Learning modular robot control policies. *IEEE Transactions on Robotics*, 39(5), 4095–4113. (Preprint *arXiv:2105.10049*.)

[56] Wolfe, K. C., Moses, M. S., Kutzer, M. D. M., & Chirikjian, G. S. (2012). M3Express: A low-cost independently-mobile reconfigurable modular robot. In *IEEE International Conference on Robotics and Automation (ICRA)* (pp. 2704–2710).

[57] Wu, L., Guo, B., Zhang, Q., Sun, Z., Zhang, J., Yu, Z., & Elkind, E. (2023). Learning to self-reconfigure for freeform modular robots via altruism proximal policy optimization. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence (IJCAI-23)* (pp. 5494–5502). https://doi.org/10.24963/ijcai.2023/610

[58] Yim, M., Roufas, K., Duff, D., Zhang, Y., Eldershaw, C., & Homans, S. (2003). Modular reconfigurable robots in space applications. *Autonomous Robots*, 14(2–3), 225–237.

[59] Yim, M., Shirmohammadi, B., Sastra, J., Park, M., Dugan, M., & Taylor, C. J. (2007). Towards robotic self-reassembly after explosion. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* (pp. 2767–2772).

[60] Yim, M., Shen, W.-M., Salemi, B., Rus, D., Moll, M., Lipson, H., Klavins, E., & Chirikjian, G. S. (2007). Modular self-reconfigurable robot systems: Grand challenges of robotics. *IEEE Robotics & Automation Magazine*, 14(1), 43–52.

[61] Yuan, Y., Song, Y., Luo, Z., Sun, W., & Kitani, K. (2022). Transform2Act: Learning a transform-and-control policy for efficient agent design. In *International Conference on Learning Representations (ICLR)*. (Preprint *arXiv:2110.03659*.)

[62] Zhang, H., Hernandez, D. E., Su, Z., & Su, B. (2018). A low cost vision-based road-following system for mobile robots. *Applied Sciences*, 8(9), 1635. https://doi.org/10.3390/app8091635

[63] Zhang, H., Li, Z., Zheng, S., Liu, Y., Zheng, P., & Zou, X. (2022). UWB/INS-based robust anchor-free relative positioning scheme for UGVs. *Measurement Science and Technology*, 33(12), 125007. https://doi.org/10.1088/1361-6501/ac8c61

[64] Zhao, D., Luo, H., Tu, Y., Meng, C., & Lam, T. L. (2024). Snail-inspired robotic swarms: A hybrid connector drives collective adaptation in unstructured outdoor environments. *Nature Communications*, 15, 3647. https://doi.org/10.1038/s41467-024-47788-2

[65] Zhu, Y., Jin, H., Zhang, X., Yin, J., Liu, P., & Zhao, J. (2014). A multi-sensory autonomous docking approach for a self-reconfigurable robot without mechanical guidance. *International Journal of Advanced Robotic Systems*, 11, 146. https://doi.org/10.5772/58867